Meredith Huang

Milestone 3 Report

[Google Drive link to the top-level m3 folder]

1. **Baseline:**
   a. **Kernel fusion**

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.499088 ms	0.37224 ms	1.450s	0.86
1000	3.9653 msms	3.53864 ms	9.301s	0.886
10000	38.6313 ms	35.1244 ms	1m29.831s	0.8714

   b. **Unfused Input Unrolling**

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	1.69971 ms	1.26255 ms	1.479s	0.86
1000	9.07503 ms	8.77193 ms	9.971s	0.886
10000	68.4671 ms	69.7446 ms	1m36.839s	0.8714

2. **Req_0: Using Streams to overlap computation with data transfer**

   [Google Drive link to subfolder req_0]

   a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

   The main idea of this optimization is to overlap data transfer and kernel execution. In the unfused baseline, each batch is processed in a standardized procedure: copying input from host to device, launching the three kernels (unroll, matmul, permute), and copy output from device back to host. All these steps happen one after another on the default stream, so if PCIe transfer slows down, the GPU sits idle during memcpy, and time would be wasted.

   With CUDA streams, we split the full batch into smaller chunks. For various chunks, we use different streams and asynchronous APIs. This ensures that when one chunk is running kernels on the GPU, the next chunk can already be copying data in or out.

   In theory, this hides part of the HtoD and DtoH latency behind computation.

   We expected the kernel op times for each layer to remain mostly unchanged. However, the total execution time went down because transfers and compute were overlapped on the timeline.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

My implementation consisted of three main steps:

i. Set up pinned host memory and CUDA streams

I created a fixed number of streams and used pinned memory for host buffers to overlap memcpy and kernel execution.

```
static const float* g_pinned_host_input = nullptr;
```

```
// Register host_input as pinned memory
g_pinned_host_input = host_input;
cudaHostRegister(const_cast<float*>(host_input), in_bytes, cudaHostRegisterDefault);

// Create streams and per-stream working buffers
cudaStream_t streams[NUM_STREAMS];
float *unrolled_buf[NUM_STREAMS];
float *matmul_buf[NUM_STREAMS];

for (int s = 0; s < NUM_STREAMS; ++s) {
    cudaStreamCreate(&streams[s]);
```

ii. Split the total batch into chunks and schedule work per chunk

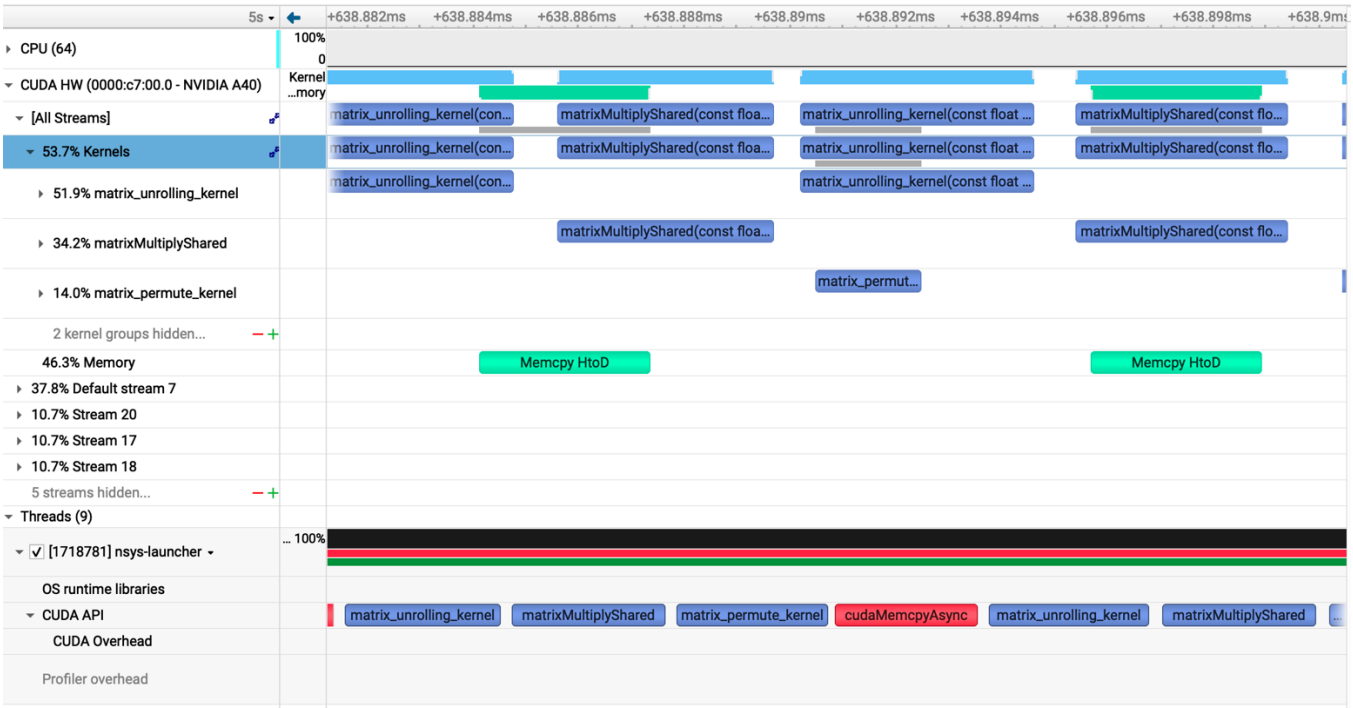
I split the batch into several chunks. Each chunk contained a consecutive range of images, and was assigned to one designated stream.

```
// Streamed processing for each image
for (int b = 0; b < Batch; ++b) {
    int s = b % NUM_STREAMS;
    cudaStream_t stream = streams[s];

    float *d_in_b  = *device_input_ptr  + (size_t)b * image_in_size;
    float *d_out_b = *device_output_ptr + (size_t)b * image_out_size;
    const float *h_in_b = host_input    + (size_t)b * image_in_size;

    // Async H2D copy
    cudaMemcpyAsync(d_in_b, h_in_b,
                    image_in_size * sizeof(float),
                    cudaMemcpyHostToDevice,
                    stream);

    // Unroll
    matrix_unrolling_kernel<<<unroll_grid, unroll_block, 0, stream>>>(
        d_in_b, unrolled_buf[s],
        1, Channel, Height, Width, K);

    // Matmul
    matrixMultiplyShared<<<matmul_grid, matmul_block, 0, stream>>>(
        *device_mask_ptr, unrolled_buf[s], matmul_buf[s],
        Map_out, Height_unrolled,
        Height_unrolled, Width_unrolled,
        Map_out, Width_unrolled);

    // Permute
    matrix_permute_kernel<<<permute_grid, PERMUTE_BLOCK_SIZE, 0, stream>>>(
        matmul_buf[s], d_out_b,
        Map_out, 1, image_pixels_out);
}
```

iii. Synchronize streams and clean up

After scheduling all chunks, I waited for all streams to finish and then destroyed them.

```
cudaDeviceSynchronize();

// Cleanup stream buffers
for (int s = 0; s < NUM_STREAMS; ++s) {
    cudaFree(unrolled_buf[s]);
    cudaFree(matmul_buf[s]);
    cudaStreamDestroy(streams[s]);
}
```



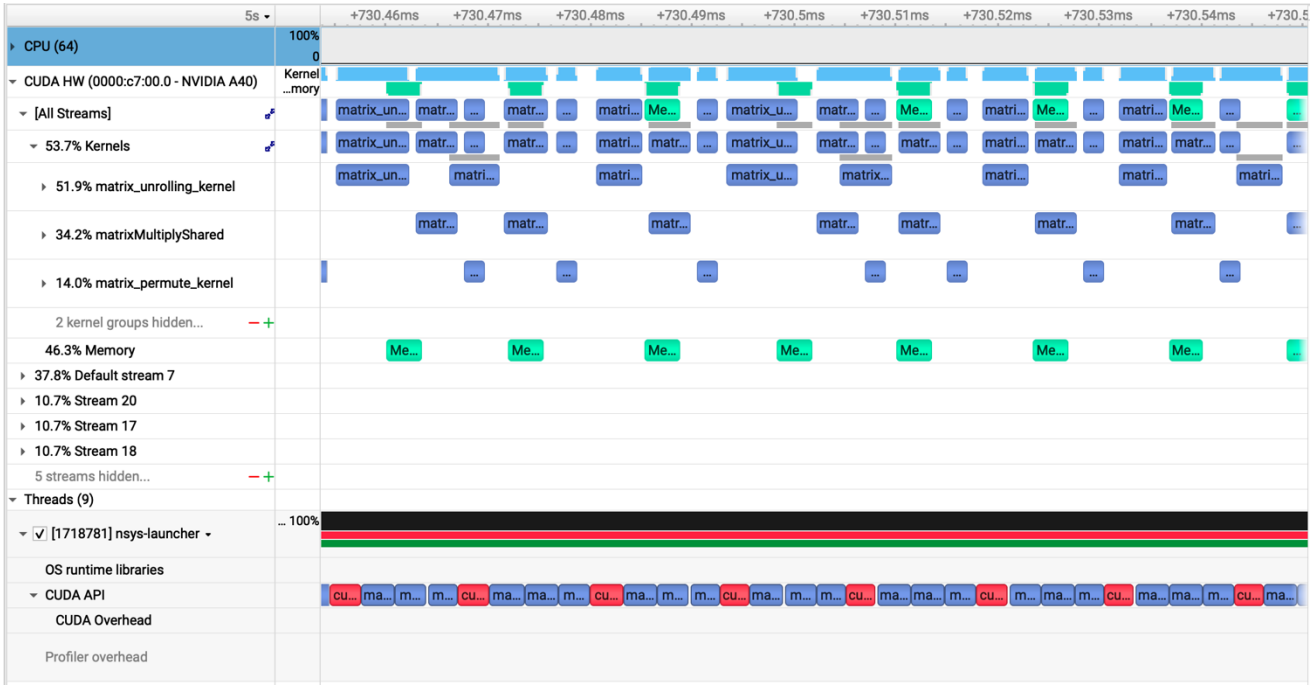
From Nsight Systems, we can see that there are multiple stream rows, memcpy, and kernels for different overlapping chunks on the timeline.

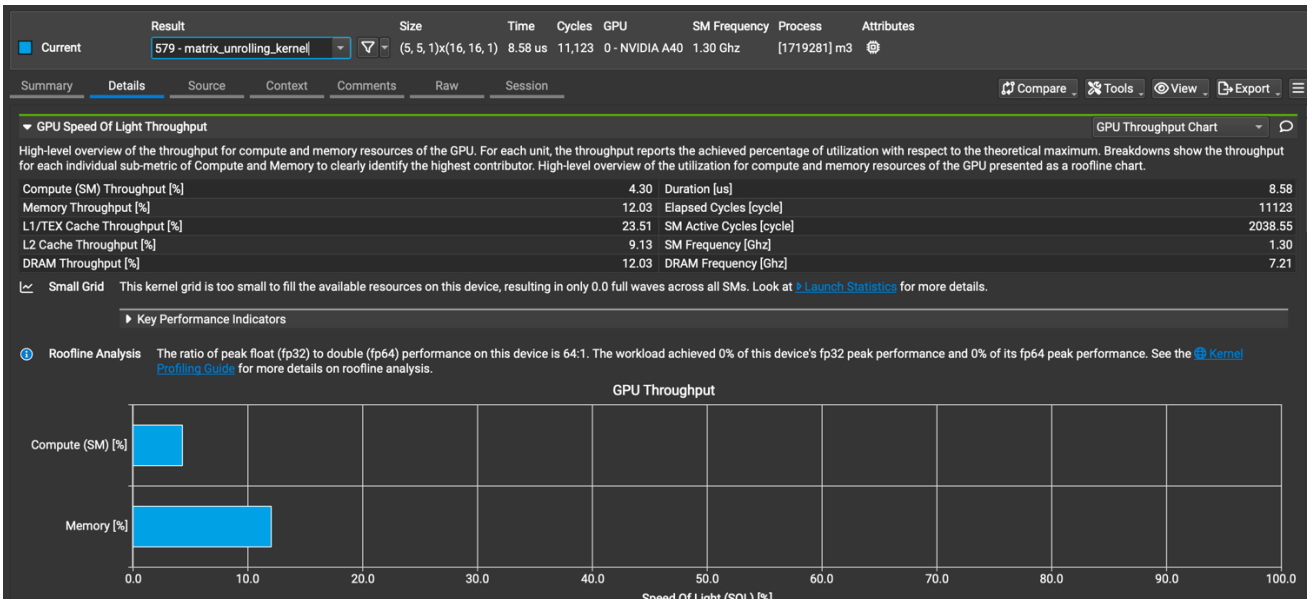
c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

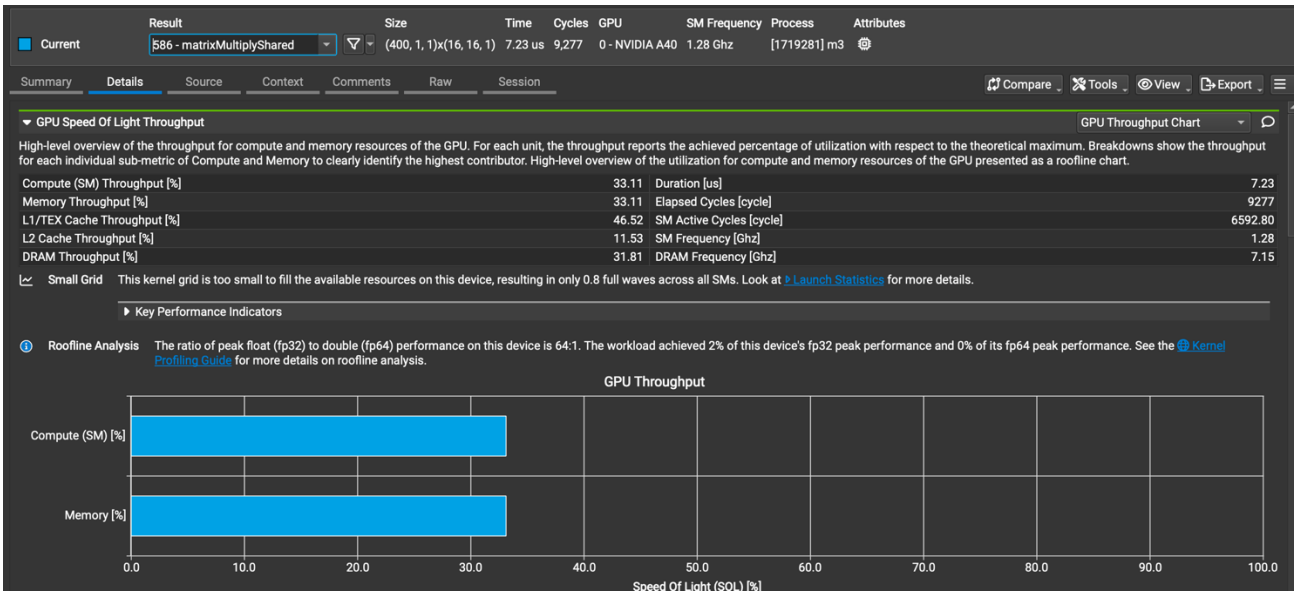
Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.000961 ms	0.001283 ms	1.468s	0.86
1000	0.000951 ms	0.001263 ms	9.452s	0.886
10000	0.001132 ms	0.001362 ms	1m31.140s	0.8714

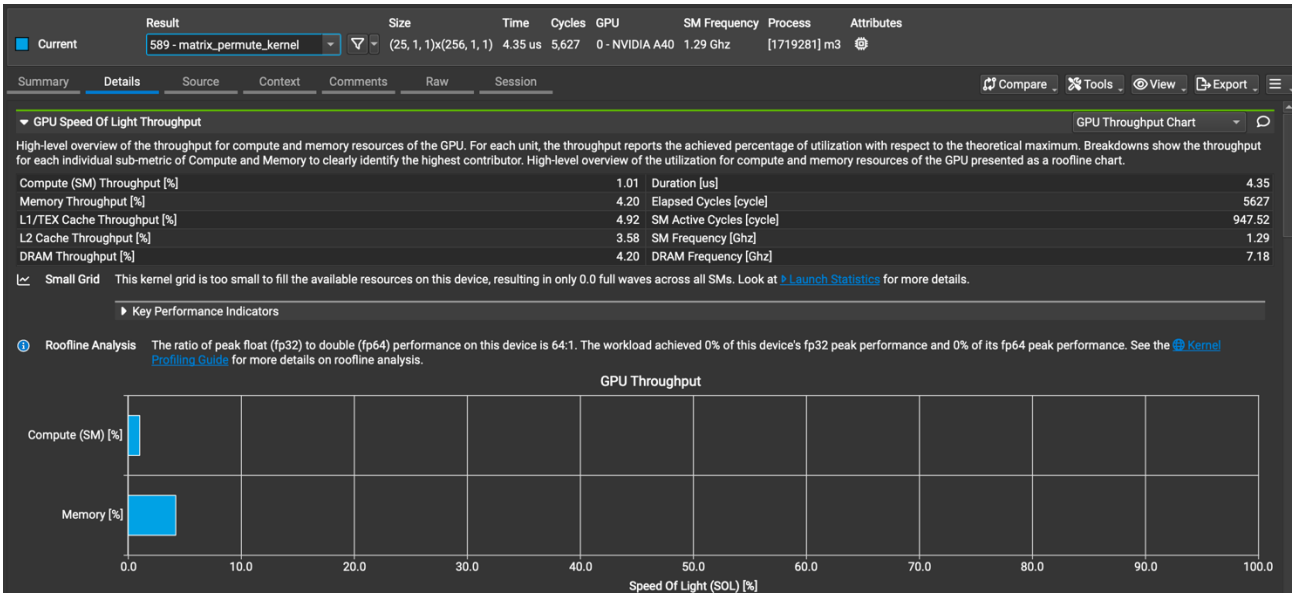
The operation time was slightly inaccurate because I placed the entire stream workload into the GPUInterface::conv_forward_gpu_prolog function. However, as

shown in the stream profiling timeline, there was some overlap between data transfer and kernel execution, which successfully achieved the intended performance improvement for stream optimization.









d. Does this optimization synergize with any other optimizations? How?

Yes. The streams work well with most of the other optimizations. While the stream splits the task into smaller pieces, the operations (such as unrolling, matrix multiplication, and permuting) remain unchanged. Therefore, inside each chunk, we can still use optimizations such as Loop unrolling, __restrict__, cuBLAS, Tensor Cores, and etc…

The idea is that per-chunk optimizations make each kernel faster, and streams make use of the time when data is moving between host and device. As a result, they reduce both the compute time and the visible transfer time. This shortens the total execution time compared to using only one of them.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)
   i. Slides from Lecture 20
   ii. Chapter 6.2.8.5 of the CUDA C++ Programming Guide
   iii. https://developer.nvidia.com/blog/how-overlap-data-transfers-cuda-cc/

3. Req_1: Using Tensor Cores to speed up matrix multiplication

[Google Drive link to subfolder req_1]

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

The idea of this optimization is to replace the normal FMA-based matrix multiplication with Tensor Cores using the WMMA API.
We treat convolution as a big matrix multiplication as usual, but instead of doing this with standard FP32 ALUs, we ask Tensor Cores to do the main GEMM part in mixed precision (TF32 input with FP32 accumulation).

In theory, Tensor Cores can complete a lot more floating-point operations per cycle than normal CUDA cores, especially for 16×16 tiles. This means if our matrices are large enough, dimensions align reasonably with 16×16 tiles, and memory traffic isn't the main bottleneck, we can expect the matrix multiplication part of convolution to be faster, especially for large batch sizes.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.
   i. I didn't change the unrolling layout or the mapping from (m, h_out, w_out, b) to linear indices. This ensured that the Tensor Core version and the baseline version

were computing the same mathematical result. For each K-tile along the reduction dimension, I loaded a 16 × 16 patch of A and B from global memory into shA and shB. Tensor Core TF32 WMMA operation works on a 16 × 16 × 16 tile, with a 256-byte load limit. Knowing this, I split each 16×16 K-slice into two pieces in shared memory, then packed them into four small shared buffers.

```
__shared__ float shA[TILE_WIDTH * TILE_WIDTH];
__shared__ float shB[TILE_WIDTH * TILE_WIDTH];

__shared__ float shA0[WMMA_M * WMMA_K];
__shared__ float shA1[WMMA_M * WMMA_K];
__shared__ float shB0[WMMA_K * WMMA_N];
__shared__ float shB1[WMMA_K * WMMA_N];
```

```
// pack A: shA -> shA0 (cols 0..7) and shA1 (cols 8..15)
if (tx < WMMA_K) {
    shA0[ty * WMMA_K + tx] = shA[ty * TILE_WIDTH + tx];
    shA1[ty * WMMA_K + tx] = shA[ty * TILE_WIDTH + (tx + WMMA_K)];
}

// pack B: shB -> shB0 (rows 0..7) and shB1 (rows 8..15)
if (ty < WMMA_K) {
    shB0[ty * WMMA_N + tx] = shB[ty * TILE_WIDTH + tx];
} else {
    int ty2 = ty - WMMA_K;
    shB1[ty2 * WMMA_N + tx] = shB[ty * TILE_WIDTH + tx];
}
```

ii. Convert shared tiles to TF32 WMMA fragments and run mma_sync

After loading one 16×16 tile of A and B into shared memory, I used WMMA to complete two Tensor Core multiplications per K-tile. The loop over all K-tiles kept accumulating into acc_frag.

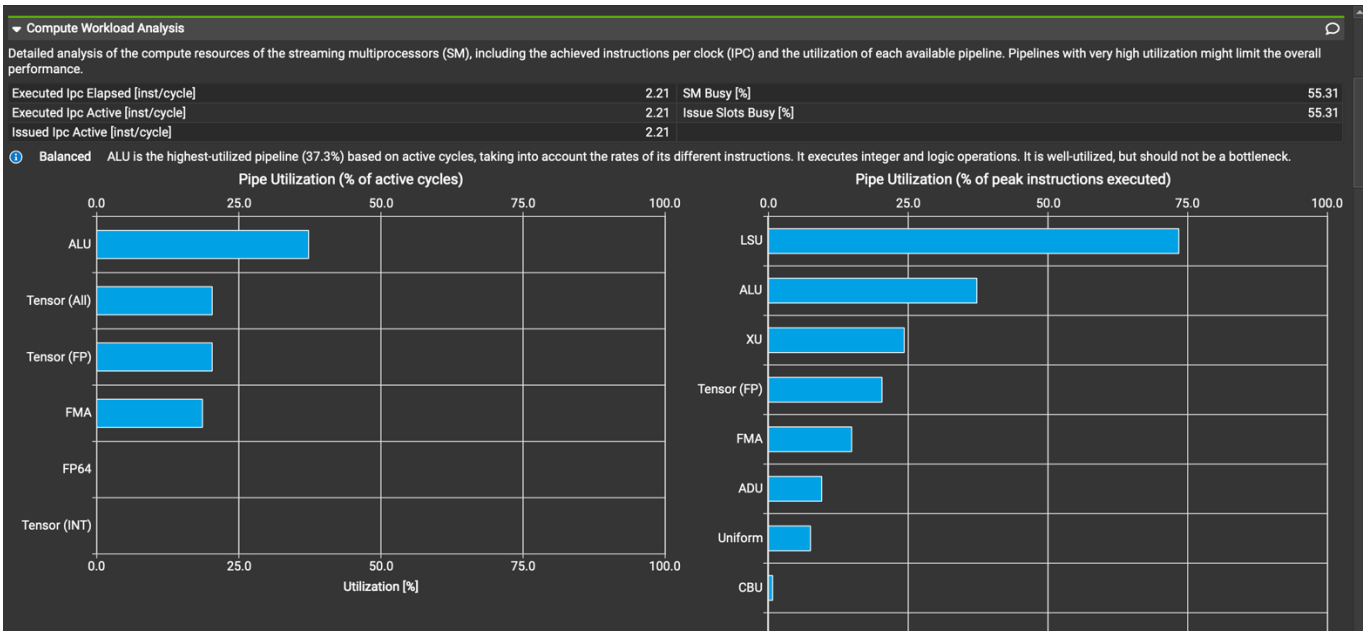
```
// WMMA fragments (TF32 inputs, FP32 accumulator)
wmma::fragment<wmma::matrix_a, WMMA_M, WMMA_N, WMMA_K,
               wmma::precision::tf32, wmma::row_major> a_frag;
wmma::fragment<wmma::matrix_b, WMMA_M, WMMA_N, WMMA_K,
               wmma::precision::tf32, wmma::row_major> b_frag;
wmma::fragment<wmma::accumulator, WMMA_M, WMMA_N, WMMA_K, float> acc_frag;

wmma::fill_fragment(acc_frag, 0.0f);
```

```
// First K-slice: A0 (16x8) * B0 (8x16)
wmma::load_matrix_sync(a_frag, shA0, WMMA_K);
wmma::load_matrix_sync(b_frag, shB0, WMMA_N);
wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);

// Second K-slice: A1 (16x8) * B1 (8x16)
wmma::load_matrix_sync(a_frag, shA1, WMMA_K);
wmma::load_matrix_sync(b_frag, shB1, WMMA_N);
wmma::mma_sync(acc_frag, a_frag, b_frag, acc_frag);
```

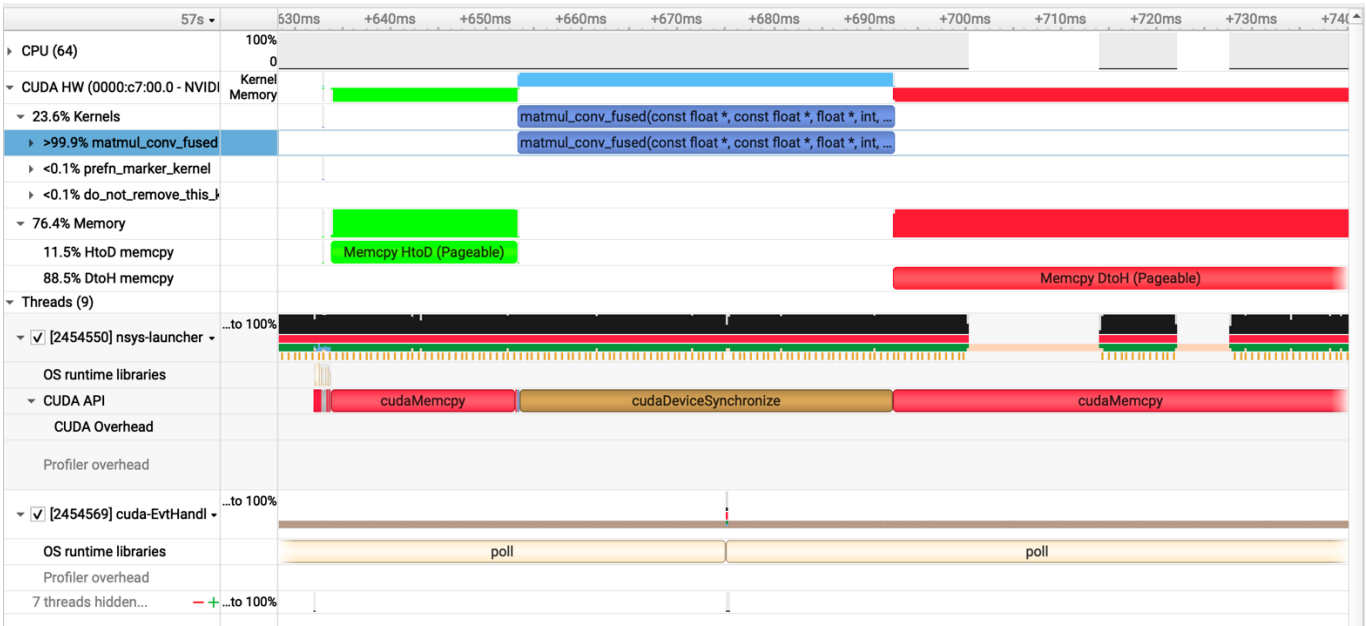
iii. Store tiles and write results back to global memory

After finishing all tiles along H_unroll, I stored the WMMA accumulator back to shared memory, then wrote it to the output in global memory.

```
// store 16x16 C tile back to shared memory, row-major
wmma::store_matrix_sync(shC, acc_frag, WMMA_N, wmma::mem_row_major);
__syncthreads();

// write results from shC to global output in NCHW layout
if (row < numCRows && col < numCColumns) {
    int b  = col / (H_out * W_out);
    int hw = col % (H_out * W_out);
    int h  = hw / W_out;
    int w  = hw - h * W_out;

    float val = shC[ty * WMMA_N + tx];
    output[b * Map_out * H_out * W_out +
           row * H_out * W_out +
           h * W_out + w] = val;
}
```



Tensor cores accelerate computation by handling mixed-precision FP16 operations, as shown by 0% FP32/FP64 usage, confirming that they offload work from traditional ALUs.



We can see from the profiler that *matmul_conv_fused* is the dominant hotspot, occupying more than 99.99% of the total GPU time. This aligns with the expected behavior, since the whole convolution workload is concentrated inside this fused matrix-multiplication kernel.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

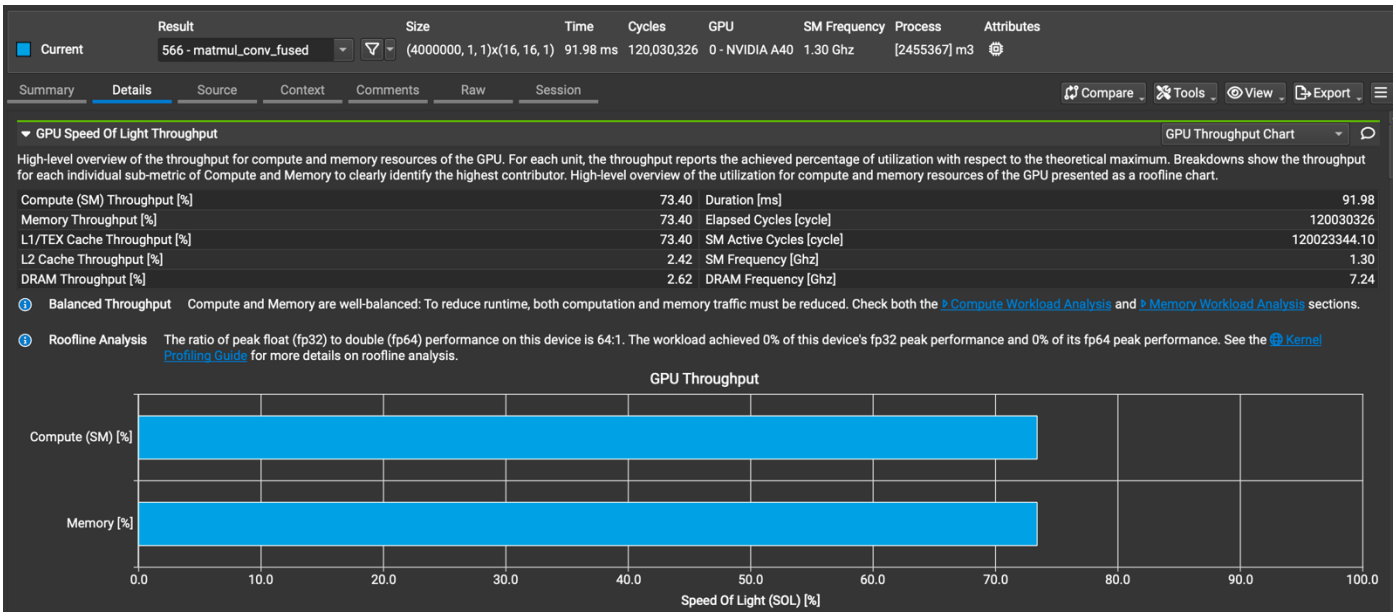
Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.794029 ms	0.407183 ms	1.372s	0.86
1000	7.00934 ms	3.93419 ms	9.228s	0.886
10000	69.8292 ms	39.0694 ms	1m28.227s	0.8714

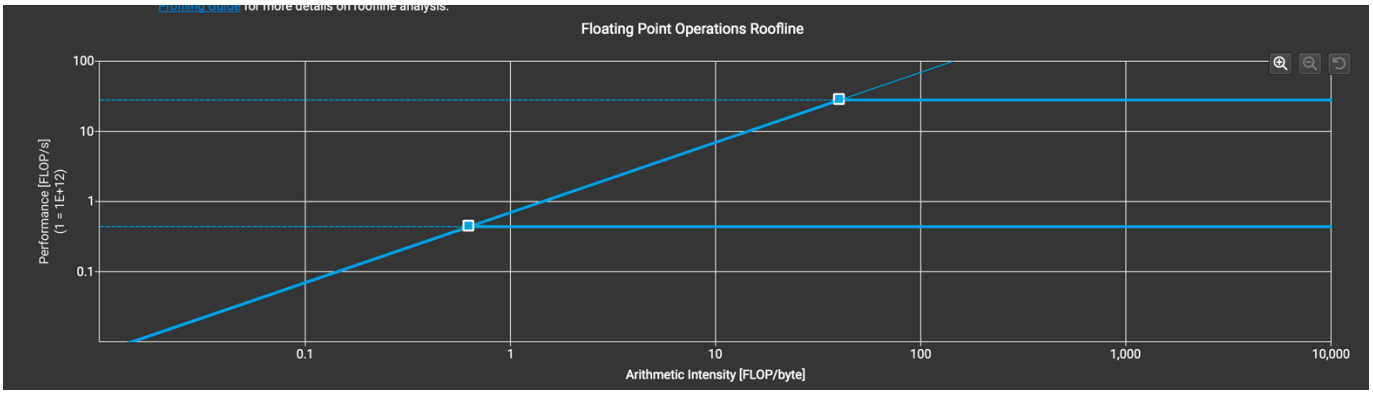
No, the result did not fully match the ideal expectation. Per-layer op times were larger than the baseline, but the total execution time was slightly better. The overall runtime improved a little (around 1–2%), even though the per-layer op times increased.

One of the possible reasons might be because the matrix sizes weren't perfectly aligned with the 16×16 tiles, so some Tensor Core work was wasted on padded zeros. Moreover, there was extra overhead from loading tiles into shared memory, rearranging them into the format expected by WMMA, calling mma_sync, and then storing the results back. As a result, the Tensor Core kernel became heavier than the simple fused kernel, and op time increased.

The small gain in total runtime was likely due to small measurement differences and slightly different balance between computation and other parts of the pipeline, rather than a big Tensor Core speedup.

Overall, this optimization was correct and numerically stable, but the performance gains were small and the per-layer kernel was actually slower than the baseline.





d. Does this optimization synergize with any other optimizations? How?

   Yes. The matrix multiplication in the kernel fusion optimization could be further enhanced by incorporating cuBLAS or TensorCore. Additionally, we can also implement streaming to overlap kernel execution with data transfer.

   Because Tensor Cores optimize the math part of GEMM, other optimizations that reduce memory cost or loop overhead can still be applied around this core to improve overall performance.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

   i. Slides from Lecture 22

   ii. Chapter 10.24.1 of the CUDA C++ Programming Guide

   iii. https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9

iv. https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html - warp-matrix-functions

4. Op_0: Weight matrix (Kernel) in constant memory

[Google Drive link to subfolder op_0]

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

Constant memory is designed for small read-only data that is broadcast to many threads. In our convolution, the mask weights are the same for every output pixel, so each thread block keeps reading the same values repeatedly.

By putting the mask into constant memory instead of global memory, mask loads should be faster, L1/L2 traffic should decrease, and total global memory pressure should drop.

The expected behavior is a reduction in memory load overhead for the mask, especially when many threads read the same element at the same time.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

i. Declared a global constant array to hold the convolution mask

```
__constant__ float const_mask[MAX_CONST_MASK_ELEMS];
```

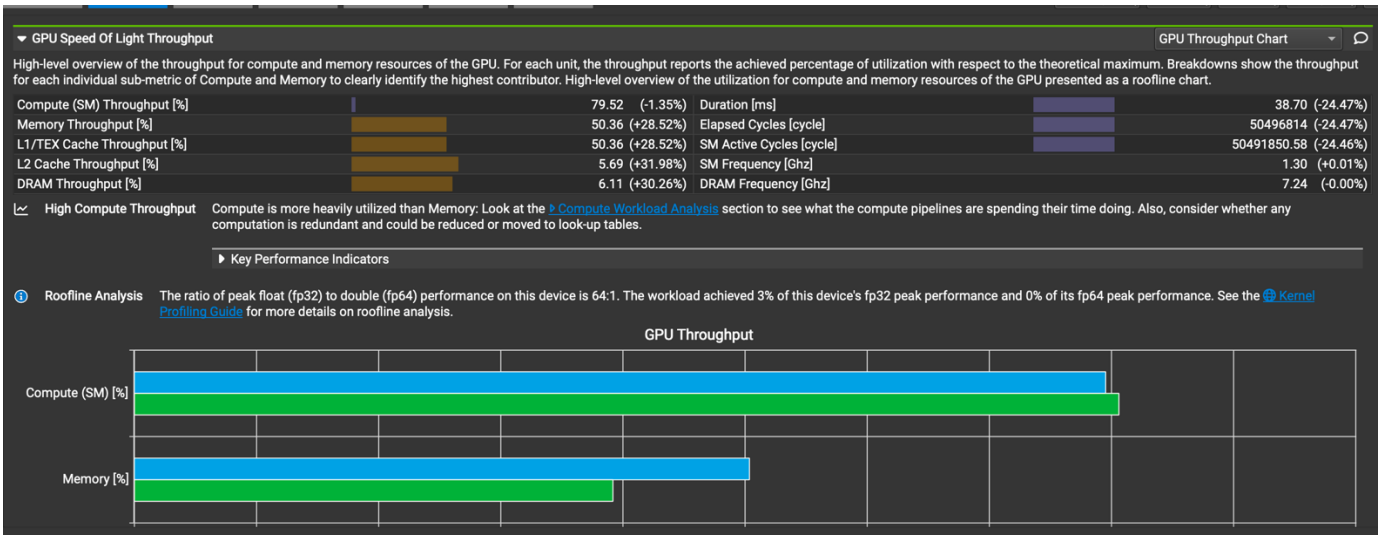
ii. Copy the host mask into constant memory

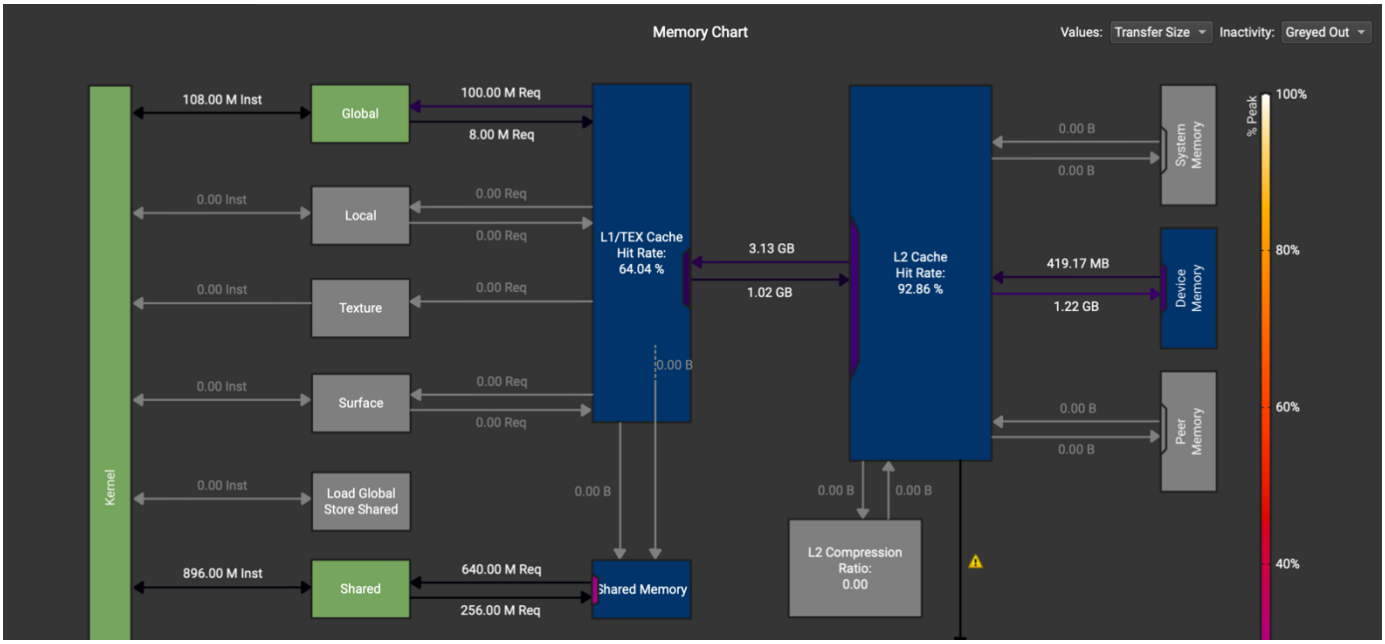
```
// copy kernel weights into constant memory
cudaError_t err = cudaMemcpyToSymbol(const_mask, host_mask, mask_size, 0, cudaMemcpyHostToDevice);
```

iii. Replaced all mask accesses with constant memory reads inside the fused matmul kernel

This ensures the mask is fetched from constant memory instead of global memory during every tile load.

```
#define MASK4D(m, c, p, q) const_mask[(m) * (Channel * K * K) + (c) * (K * K) + (p) * K + (q)]
```





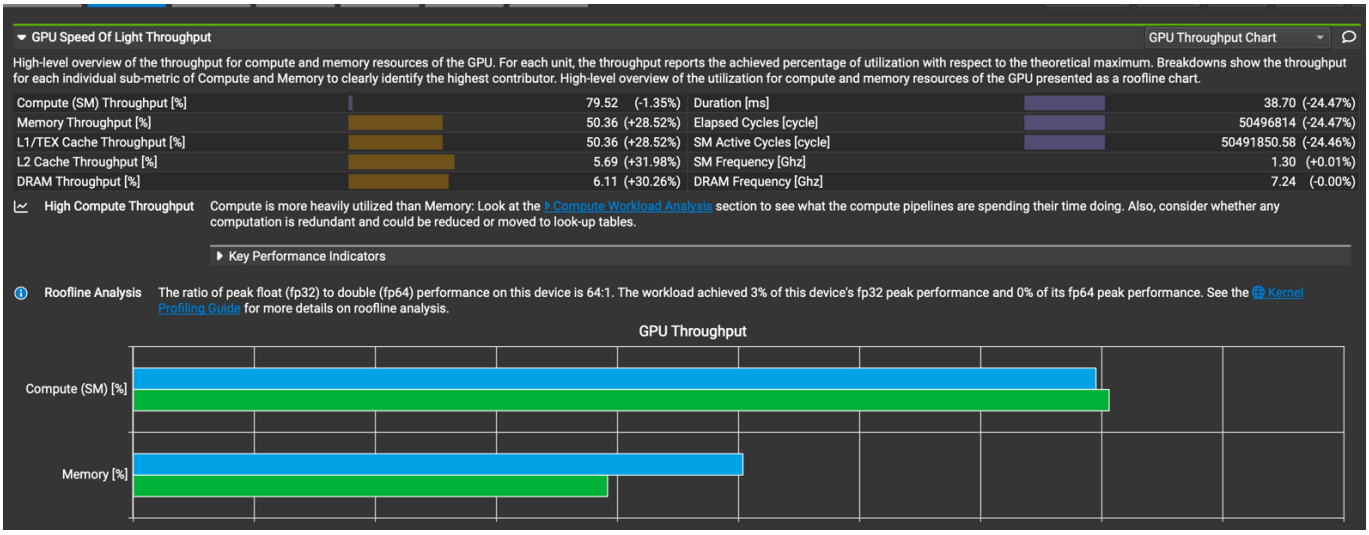
From profiling, we can see that global memory read transactions dropped and constant memory loads appeared inside the memory section. This matches the theoretical behavior of constant memory.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

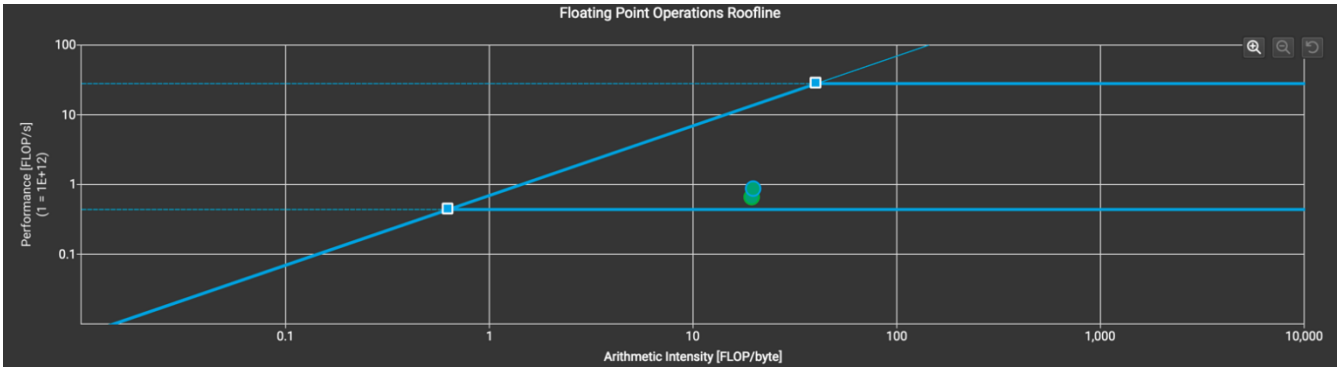
Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.320972 ms	0.487794 ms	1.468s	0.86
1000	2.93537 ms	4.77417 ms	9.432s	0.886
10000	29.3536 ms	47.5724 ms	1m30.855s	0.8714

Yes, the results partially match expectations.

- OP1 time became faster for all batch sizes -> this makes sense because mask fetches became lighter
- OP2 became slower -> this suggests the bottleneck for the second convolution layer is more compute-heavy, and constant memory does not help in that case
- Total execution time is very similar to the baseline -> despite lower memory pressure, kernel is still compute-heavy



We can also see that SM remained the same throughput, while memory throughput changed only slightly. This means the kernel's runtime is dominated by matrix multiply and unrolled input loads, not by mask fetching. Therefore, constant memory helps a small part of the kernel, but the overall effect is limited.



d. Does this optimization synergize with any other optimizations? How?

Yes. Constant memory does not conflict with other optimizations because the mask is read-only and extremely small. Streams also work fine, because mask only needs to be copied once before launching all kernels.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

i. Slides from Lecture 7
ii. Chapter 10.2.2 of the CUDA C++ Programming Guide

5. Op_1: __restrict__ keyword

[Google Drive link to subfolder op_1]

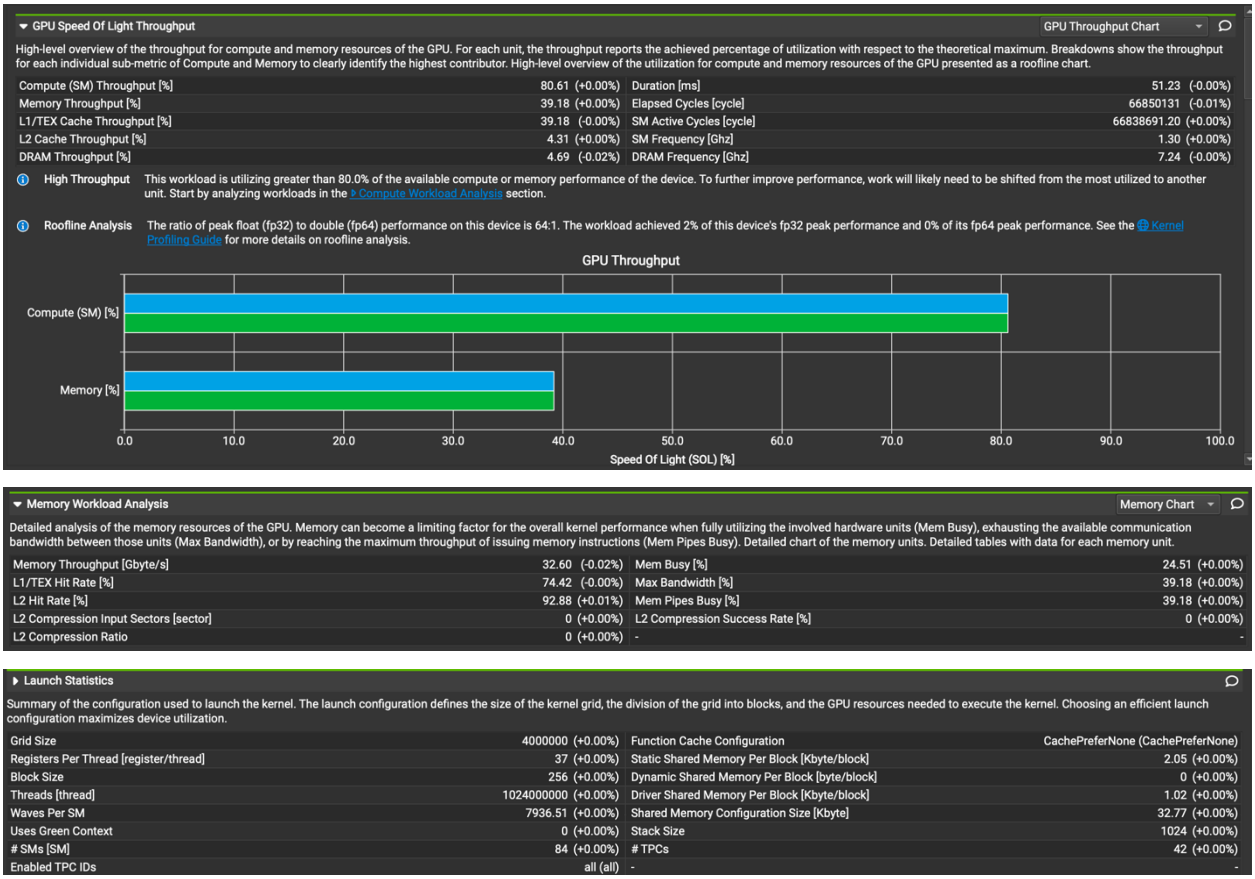
a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

The __restrict__ keyword tells the compiler that different pointers do not alias the same memory region. When the compiler knows this, it is allowed to assume each pointer refers to independent memory, reduce unnecessary reloads, schedule memory instructions more aggressively, and avoid inserting conservative alias checks.

The expected behavior is a small improvement in memory-related efficiency. But in many CUDA kernels, especially matrix-unrolling + GEMM kernels, the improvement can be minimal because the pointers already come from separate cudaMalloc buffers.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.
I added the __restrict__ keyword to all CUDA kernel pointer arguments that should be treated as non-aliasing. These three buffers in fused convolution kernel are allocated separately with cudaMalloc, and therefore genuinely do not alias.

```
__global__ void matmul_conv_fused(const float *__restrict__ mask, const float *__restrict__ input, float *__restrict__ output,
                                  int Batch, int Map_out, int Channel, int Height, int Width, int K)
```

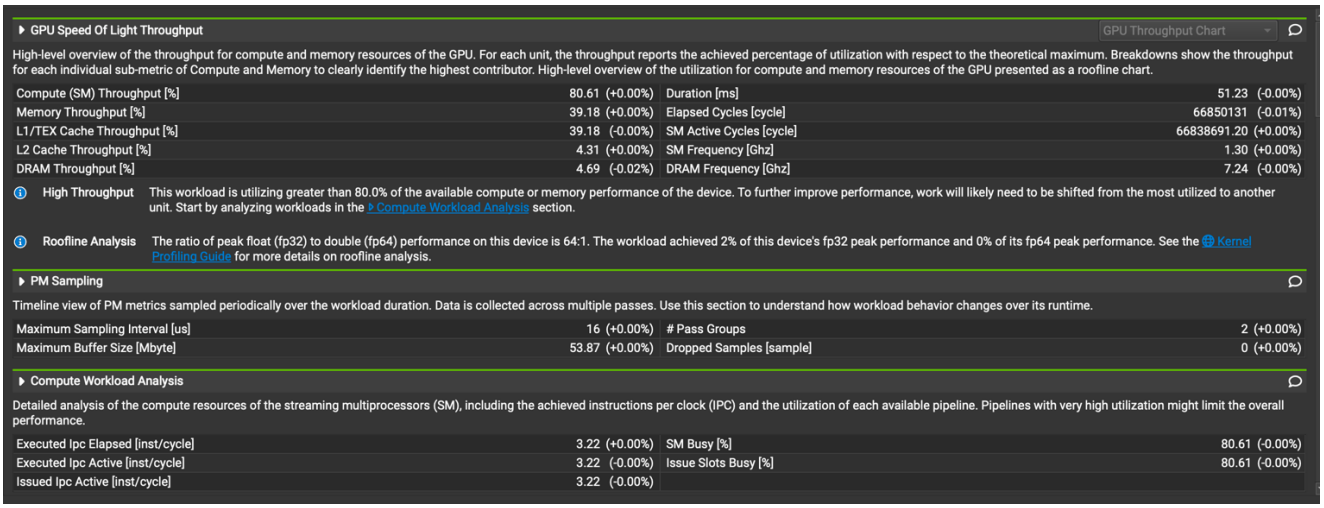


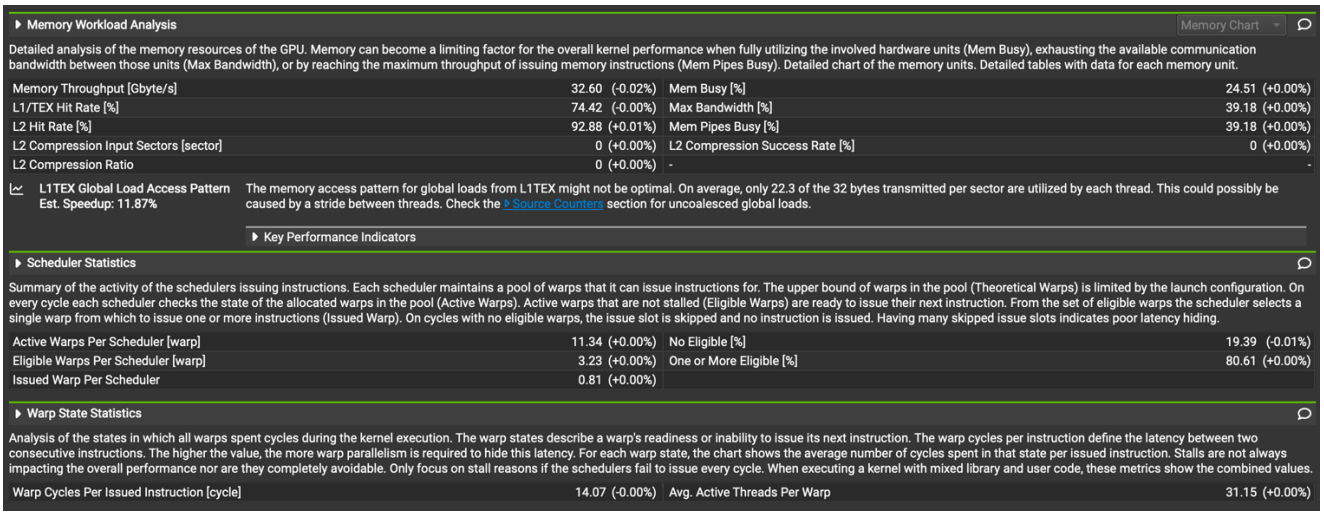
From Nsight Compute, we can see SM throughput, memory transactions stayed almost the same, and register usage did not change. Also, no new stalls were introduced. This matches the expected behavior.

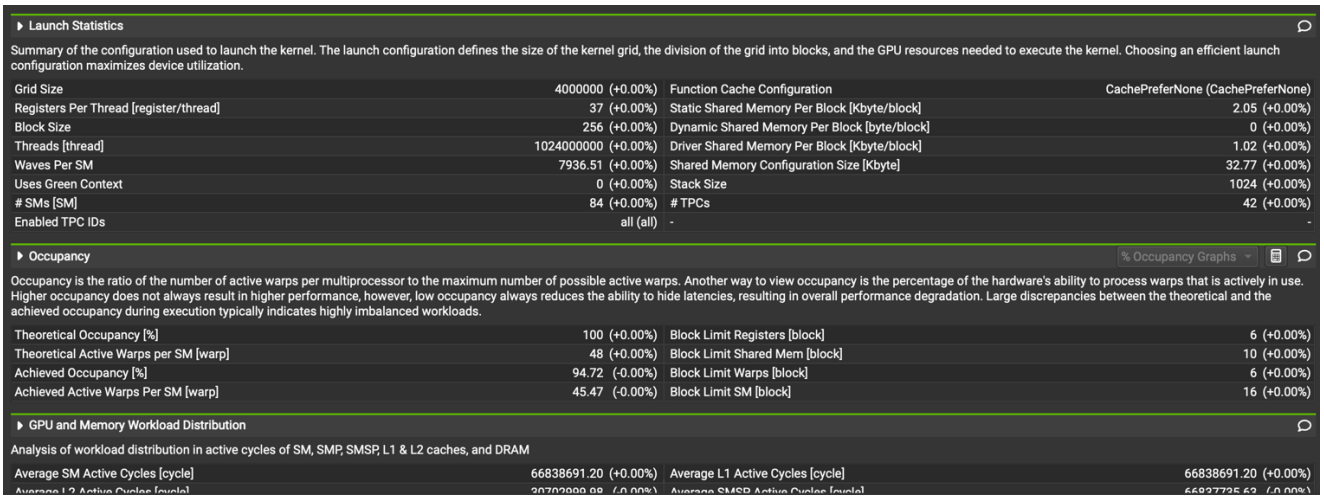
c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.495198 ms	0.369833 ms	1.483s	0.86
1000	3.94018 ms	3.53055 ms	9.171s	0.886
10000	38.9157 ms	35.0703 ms	1m28.473s	0.8714

Yes, the performance behaves as expected. There's a small improvements in OP1 and OP2, but mostly within normal noise levels. In addition, total runtime remains roughly the same and accuracy remains unchanged.







▸ Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	4000000 (+0.00%)	Function Cache Configuration	CachePreferNone (CachePreferNone)
Registers Per Thread [register/thread]	37 (+0.00%)	Static Shared Memory Per Block [Kbyte/block]	2.05 (+0.00%)
Block Size	256 (+0.00%)	Dynamic Shared Memory Per Block [byte/block]	0 (+0.00%)
Threads [thread]	1024000000 (+0.00%)	Driver Shared Memory Per Block [Kbyte/block]	1.02 (+0.00%)
Waves Per SM	7936.51 (+0.00%)	Shared Memory Configuration Size [Kbyte]	32.77 (+0.00%)
Uses Green Context	0 (+0.00%)	Stack Size	1024 (+0.00%)
# SMs [SM]	84 (+0.00%)	# TPCs	42 (+0.00%)
Enabled TPC IDs	all (all)	-	-

▸ Occupancy

% Occupancy Graphs

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100 (+0.00%)	Block Limit Registers [block]	6 (+0.00%)
Theoretical Active Warps per SM [warp]	48 (+0.00%)	Block Limit Shared Mem [block]	10 (+0.00%)
Achieved Occupancy [%]	94.72 (-0.00%)	Block Limit Warps [block]	6 (+0.00%)
Achieved Active Warps Per SM [warp]	45.47 (-0.00%)	Block Limit SM [block]	16 (+0.00%)

▸ GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	66838691.20 (+0.00%)	Average L1 Active Cycles [cycle]	66838691.20 (+0.00%)

From profiling, we can see that there's no major structural change.

This matches the typical result of using __restrict__ in GPU programs. Because each buffer is separately cudaMalloc'ed, the compiler likely already assumed no aliasing, so the keyword brings little extra information.

d. Does this optimization synergize with any other optimizations? How?

Yes. Since __restrict__ does not change program behavior, it works with any other optimizations.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

i. Chapter 10.2.6. of the CUDA C++ Programming Guide

ii. https://developer.nvidia.com/blog/cuda-pro-tip-optimize-pointer-aliasing/

6. Op_2: Loop unrolling

[Google Drive link to subfolder op_2]

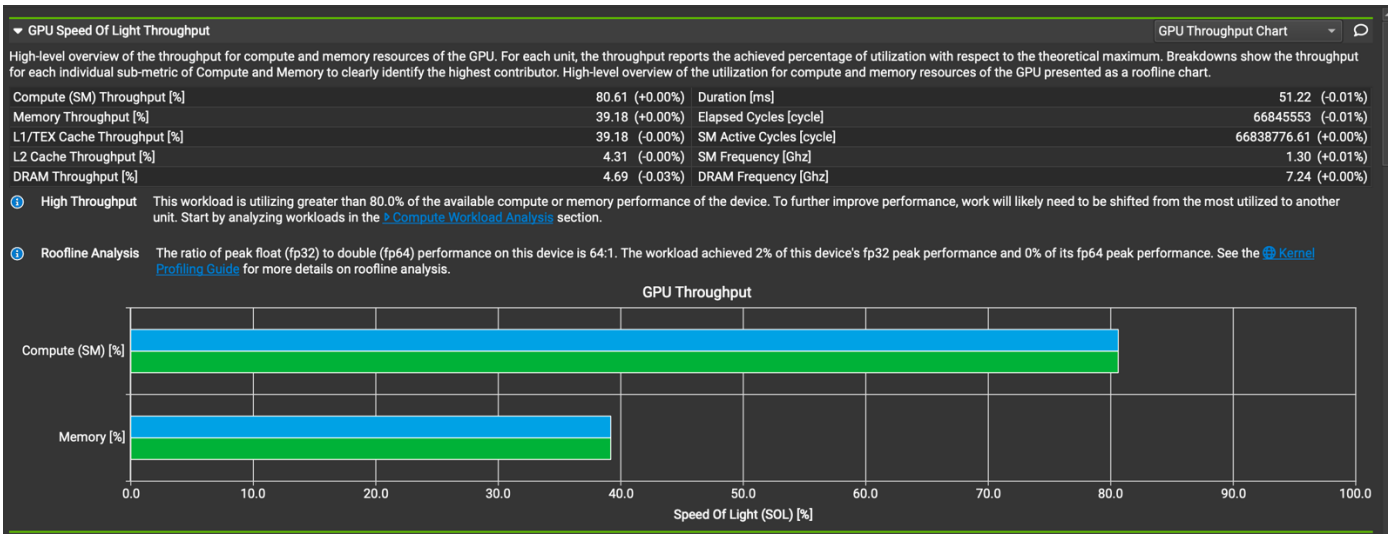
a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

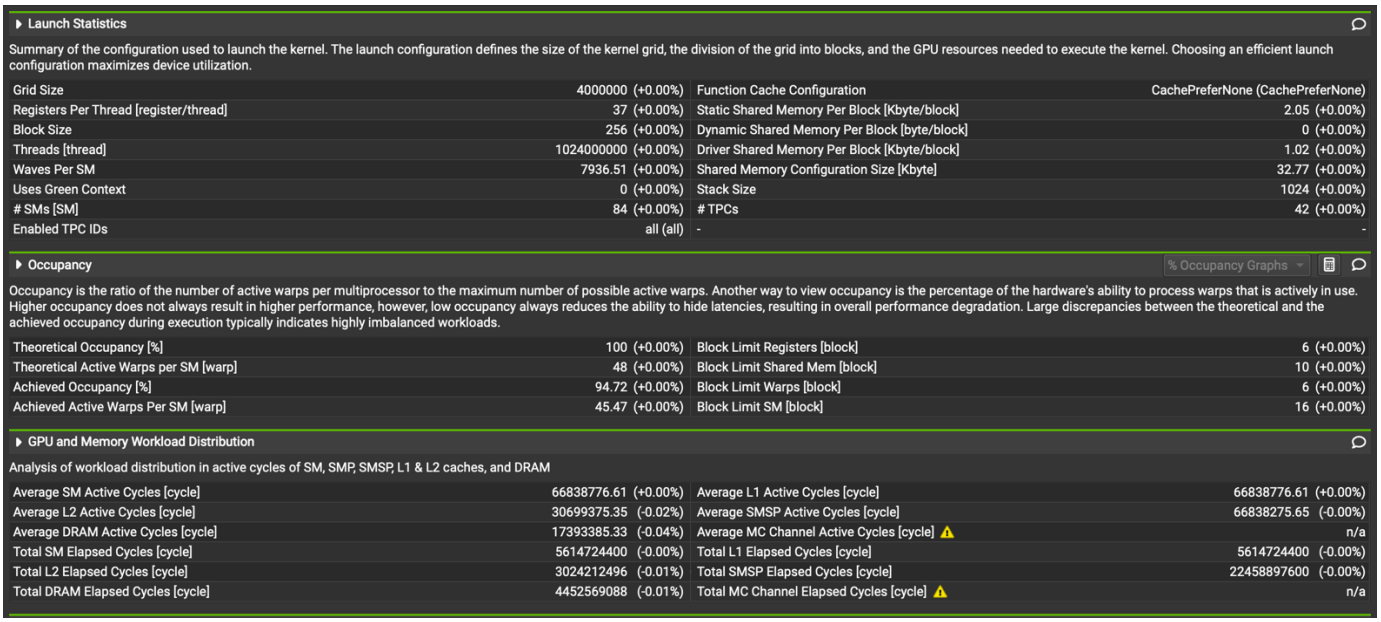
Loop unrolling expands the loop body so that more work happens in one iteration. Removing the loop control overhead can reduce the number of branch evaluations, increase instruction-level parallelism, and help the compiler schedule or pipeline operations more efficiently.

In theory, this can slightly speed up the inner product section of the convolution's matrix multiplication. But since our loop bound (TILE_WIDTH = 16) is small and known at compile time, the expected gain might be small.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.
I applied loop unrolling inside the fused matrix multiplication kernel, specifically in the inner accumulation loop where each tile contributes to the output value. This removes loop indexing overhead and allows the compiler to schedule multiply-adds more effectively.

```
if (row < numCRows && col < numCCols) {
    #pragma unroll
    for (int i = 0; i < TILE_W; i += 4) {
        acc += tile_mask[ty][i    ] * tile_input[i    ][tx];
        acc += tile_mask[ty][i + 1] * tile_input[i + 1][tx];
        acc += tile_mask[ty][i + 2] * tile_input[i + 2][tx];
        acc += tile_mask[ty][i + 3] * tile_input[i + 3][tx];
    }
}
```





Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	4000000 (+0.00%)	Function Cache Configuration	CachePreferNone (CachePreferNone)
Registers Per Thread [register/thread]	37 (+0.00%)	Static Shared Memory Per Block [Kbyte/block]	2.05 (+0.00%)
Block Size	256 (+0.00%)	Dynamic Shared Memory Per Block [byte/block]	0 (+0.00%)
Threads [thread]	1024000000 (+0.00%)	Driver Shared Memory Per Block [Kbyte/block]	1.02 (+0.00%)
Waves Per SM	7936.51 (+0.00%)	Shared Memory Configuration Size [Kbyte]	32.77 (+0.00%)
Uses Green Context	0 (+0.00%)	Stack Size	1024 (+0.00%)
# SMs [SM]	84 (+0.00%)	# TPCs	42 (+0.00%)
Enabled TPC IDs	all (all)	-	-

Occupancy — % Occupancy Graphs

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100 (+0.00%)	Block Limit Registers [block]	6 (+0.00%)
Theoretical Active Warps per SM [warp]	48 (+0.00%)	Block Limit Shared Mem [block]	10 (+0.00%)
Achieved Occupancy [%]	94.72 (+0.00%)	Block Limit Warps [block]	6 (+0.00%)
Achieved Active Warps Per SM [warp]	45.47 (+0.00%)	Block Limit SM [block]	16 (+0.00%)

GPU and Memory Workload Distribution

Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	66838776.61 (+0.00%)	Average L1 Active Cycles [cycle]	66838776.61 (+0.00%)
Average L2 Active Cycles [cycle]	30699375.35 (-0.02%)	Average SMSP Active Cycles [cycle]	66838275.65 (-0.00%)
Average DRAM Active Cycles [cycle]	17393385.33 (-0.04%)	Average MC Channel Active Cycles [cycle] ⚠	n/a
Total SM Elapsed Cycles [cycle]	5614724400 (-0.00%)	Total L1 Elapsed Cycles [cycle]	5614724400 (-0.00%)
Total L2 Elapsed Cycles [cycle]	3024212496 (-0.01%)	Total SMSP Elapsed Cycles [cycle]	22458897600 (-0.00%)
Total DRAM Elapsed Cycles [cycle]	4452569088 (-0.01%)	Total MC Channel Elapsed Cycles [cycle] ⚠	n/a

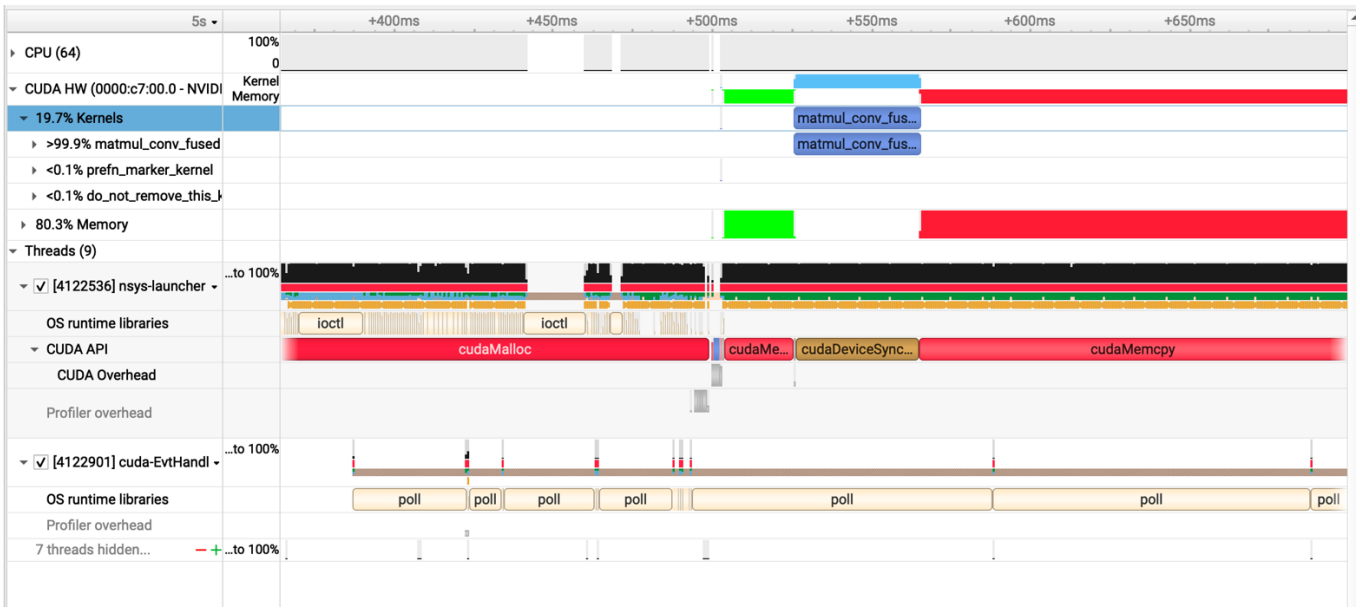
From profiling, we can see memory throughput remained the same and shows no new stalls, and register count stayed stable. This confirms the implementation is correct and does not break memory behavior.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.480449 ms	0.369191 ms	1.407s	0.86
1000	3.93852 ms	3.53716 ms	8.958s	0.886
10000	38.6165 ms	35.0894 ms	1m26.484s	0.8714

Yes, the performance aligned with expectations. There's slight improvement in op time, total runtime slightly faster and accuracy preserved entirely.

Overall, the effect is small but correct, which is typical for loop unrolling in tiled GEMM-style kernels.



d. Does this optimization synergize with any other optimizations? How?

Yes. Loop unrolling is harmless and fully compatible with other optimizations. Because it only changes arithmetic structure inside the kernel, it does not conflict with memory or scheduling optimizations.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

i. the instructions in the README_m3.md on GitHub

ii. https://forums.developer.nvidia.com/t/understanding-unrolling-and-concurrent-memory-operations/38617

7. **Op_3: Sweeping various parameters to find best values (block sizes, amount of thread coarsening) -- requires tables/graphs in Report**

[Google Drive link to subfolder op_3]

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

We're not changing the math of the convolution, instead, I only change the block size (TILE_W) and the amount of thread coarsening (THREAD_TILE).

For block size (TILE_W), because it controls how many threads are in a block and how large each shared-memory tile is. So, if the block is too small, we have high scheduling overhead and do not hide latency well. Otherwise, if the block is too large, each thread may use too many registers or shared memory, and occupancy drops.

For thread coarsening (THREAD_TILE), it lets each thread compute multiple output rows instead of only one. A thread already loads values from shared memory anyway, so letting it reuse those values for 2 rows increases arithmetic intensity. This should give better ALU utilization and reduce the number of synchronizations and shared loads per output element.

So theoretically there should be a sweet spot where occupancy is still high enough, registers per thread are not exploding, shared memory usage per block is safe, and each thread does enough useful work.
At that point I expect slightly lower op times compared to the plain fused kernel, while accuracy stays the same.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.
   i. Adding tunable parameters TILE_W and THREAD_TILE and adjusting block shape on the host

```
#ifndef TILE_W
#define TILE_W 32
#endif

#ifndef THREAD_TILE
#define THREAD_TILE 2
#endif
```

```
dim3 block(TILE_W, TILE_W / THREAD_TILE, 1);
dim3 grid((B_cols + TILE_W - 1) / TILE_W,
          (A_rows + TILE_W - 1) / TILE_W, 1);
```

   ii. Joint register + shared-memory tiling in the kernel
   Inside the kernel, each thread now owns THREAD_TILE output rows instead of one

```
int rowBase = blockIdx.y * TILE_W + ty * THREAD_TILE;
int col     = blockIdx.x * TILE_W + tx;

float acc[THREAD_TILE];
#pragma unroll
for (int r = 0; r < THREAD_TILE; ++r) {
    acc[r] = 0.f;
}
```

When loading a tile of A (mask) and B (unrolled input) into shared memory, each thread loads THREAD_TILE rows

```
for (int t = 0; t < num_tiles; ++t) {
    int kA = t * TILE_W + tx;
    int kB_base = t * TILE_W + ty * THREAD_TILE;

    #pragma unroll
    for (int r = 0; r < THREAD_TILE; ++r) {
        int row = rowBase + r;
        int shRow = ty * THREAD_TILE + r;
```

Then I reuse the same tile of Bs for all THREAD_TILE rows of As

```
if (col < num_cols) {
    for (int kInner = 0; kInner < TILE_W; ++kInner) {
        float in_val = Bs[kInner][tx];
        #pragma unroll
        for (int r = 0; r < THREAD_TILE; ++r) {
            int shRow = ty * THREAD_TILE + r;
            if (rowBase + r < num_rows) {
                acc[r] += As[shRow][kInner] * in_val;
            }
        }
    }
}
```

iii. Writes back to outputs

```
#pragma unroll
for (int r = 0; r < THREAD_TILE; ++r) {
    int row = rowBase + r;
    if (row < num_rows && col < num_cols) {
        int b  = col / image_size;
        int hw = col - b * image_size;
        int ho = hw / Wo;
        int wo = hw - ho * Wo;

        output[((b * Map_out + row) * Ho + ho) * Wo + wo] = acc[r];
    }
}
```

iv. Then compile and run the same source file with different macro values: TILE_W $\in \{8, 16, 32\}$ and THREAD_TILE $\in \{1, 2\}$.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

i. Baseline (TILE_W = 16, THREAD_TILE = 1)

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.499088 ms	0.37224 ms	1.450s	0.86
1000	3.9653 msms	3.53864 ms	9.301s	0.886
10000	38.6313 ms	35.1244 ms	1m29.831s	0.8714

ii. TILE_W = 8, THREAD_TILE = 1

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.57467 ms	0.623031 ms	1.364s	0.86
1000	4.86136 ms	6.05731 ms	9.030s	0.886
10000	47.7827 ms	60.2142 ms	1m27.148s	0.8714

iii. TILE_W = 32, THREAD_TILE = 1

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.759948 ms	0.593415 ms	1.438s	0.86
1000	6.78589 ms	5.81418 ms	9.354s	0.886
10000	67.4928 ms	57.8748 ms	1m30.516s	0.8714

iv. Baseline(TILE_W = 16) + THREAD_TILE = 2

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.615807 ms	0.329118 ms	1.547s	0.86
1000	5.31968 ms	3.10891 ms	9.210s	0.886
10000	52.4695 ms	30.8562 ms	1m28.925s	0.8714

v. TILE_W = 8 + THREAD_TILE = 2

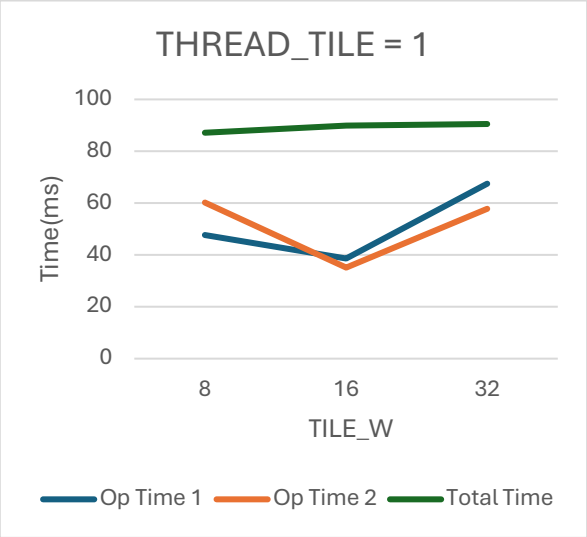
Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.550664 ms	0.598815 ms	1.482s	0.86
1000	4.54322 ms	5.8575 ms	9.025s	0.886
10000	44.56 ms	58.1864 ms	1m27.022s	0.8714

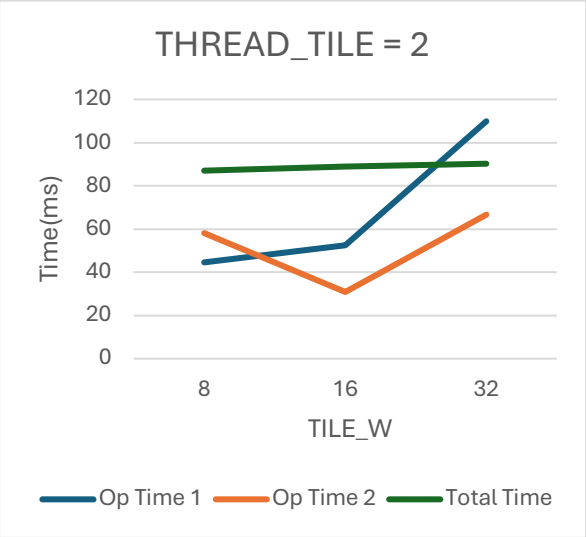
vi. TILE_W = 32 + THREAD_TILE = 2

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	1.19348 ms	0.693814 ms	1.380s	0.86
1000	10.9946 ms	6.69534 ms	9.362s	0.886
10000	109.887 ms	66.6505 ms	1m30.232s	0.8714

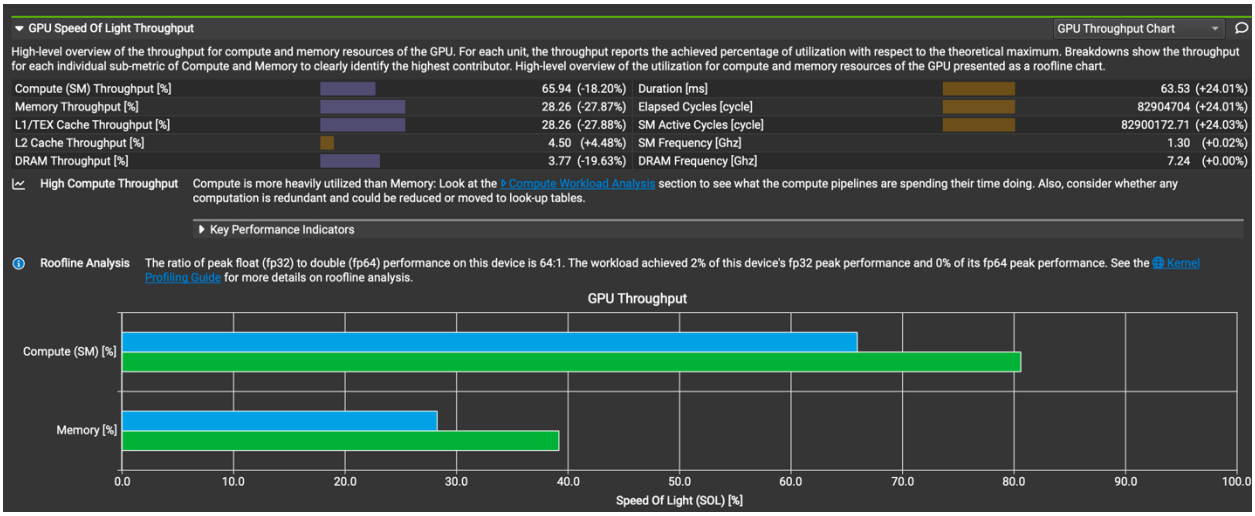
vii. Compare (batch_size = 10000)

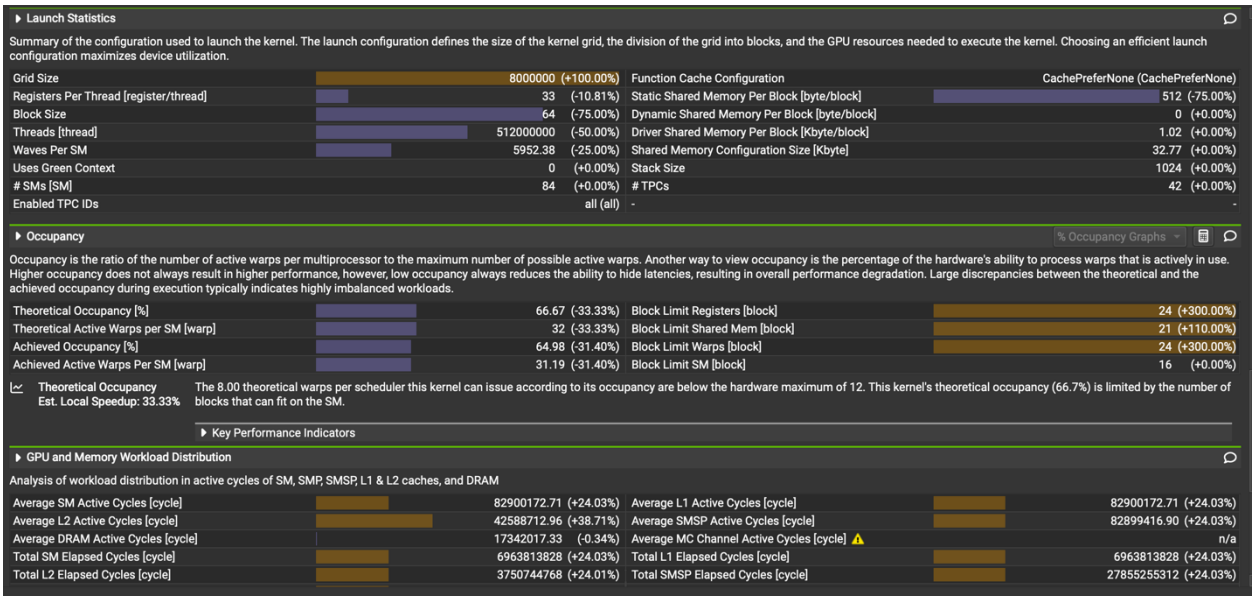
THREAD_TILE	TILE_W	Op Time 1	Op Time 2	Total Execution Time	Accuracy
1	8	47.7827 ms	60.2142 ms	1m27.148s	0.8714
	16	38.6313 ms	35.1244 ms	1m29.831s	0.8714
	32	67.4928 ms	57.8748 ms	1m30.516s	0.8714
2	8	44.56 ms	58.1864 ms	1m27.022s	0.8714
	16	52.4695 ms	30.8562 ms	1m28.925s	0.8714
	32	109.887 ms	66.6505 ms	1m30.232s	0.8714





Overall, the sweep behaved mostly as expected.





▸ Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

Grid Size	8000000 (+100.00%)	Function Cache Configuration	CachePreferNone (CachePreferNone)
Registers Per Thread [register/thread]	33 (-10.81%)	Static Shared Memory Per Block [byte/block]	512 (-75.00%)
Block Size	64 (-75.00%)	Dynamic Shared Memory Per Block [byte/block]	0 (+0.00%)
Threads [thread]	512000000 (-50.00%)	Driver Shared Memory Per Block [Kbyte/block]	1.02 (+0.00%)
Waves Per SM	5952.38 (-25.00%)	Shared Memory Configuration Size [Kbyte]	32.77 (+0.00%)
Uses Green Context	0 (+0.00%)	Stack Size	1024 (+0.00%)
# SMs [SM]	84 (+0.00%)	# TPCs	42 (+0.00%)
Enabled TPC IDs	all (all)	-	-

▸ Occupancy

% Occupancy Graphs

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	66.67 (-33.33%)	Block Limit Registers [block]	24 (+300.00%)
Theoretical Active Warps per SM [warp]	32 (-33.33%)	Block Limit Shared Mem [block]	21 (+110.00%)
Achieved Occupancy [%]	64.98 (-31.40%)	Block Limit Warps [block]	24 (+300.00%)
Achieved Active Warps Per SM [warp]	31.19 (-31.40%)	Block Limit SM [block]	16 (+0.00%)

Theoretical Occupancy
Est. Local Speedup: 33.33%

The 8.00 theoretical warps per scheduler this kernel can issue according to its occupancy are below the hardware maximum of 12. This kernel's theoretical occupancy (66.7%) is limited by the number of blocks that can fit on the SM.

▸ Key Performance Indicators

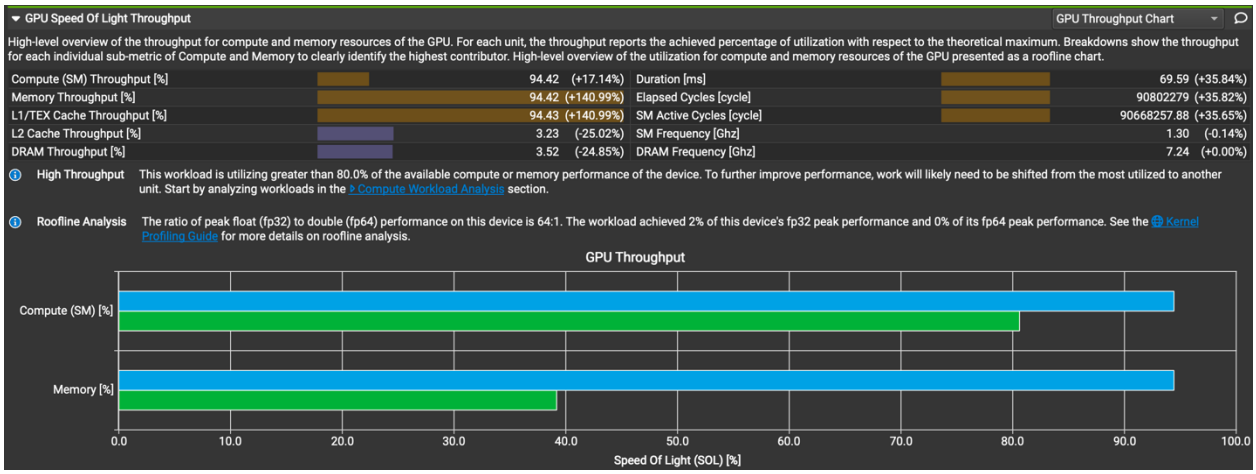
▸ GPU and Memory Workload Distribution

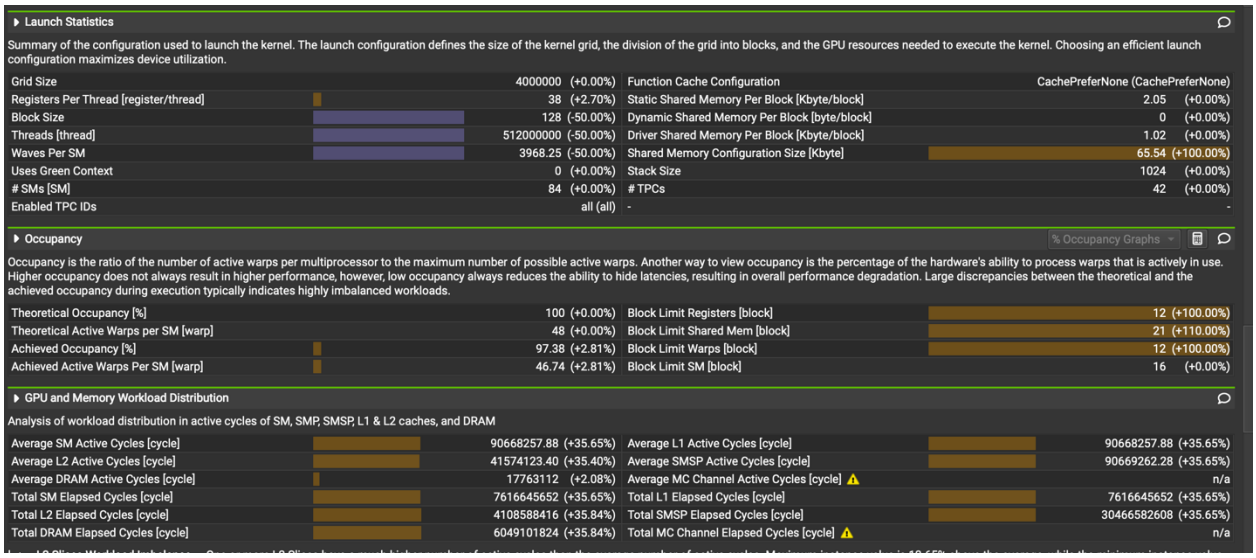
Analysis of workload distribution in active cycles of SM, SMP, SMSP, L1 & L2 caches, and DRAM

Average SM Active Cycles [cycle]	82900172.71 (+24.03%)	Average L1 Active Cycles [cycle]	82900172.71 (+24.03%)
Average L2 Active Cycles [cycle]	42588712.96 (+38.71%)	Average SMSP Active Cycles [cycle]	82899416.90 (+24.03%)
Average DRAM Active Cycles [cycle]	17342017.33 (-0.34%)	Average MC Channel Active Cycles [cycle] ⚠	n/a
Total SM Elapsed Cycles [cycle]	6963813828 (+24.03%)	Total L1 Elapsed Cycles [cycle]	6963813828 (+24.03%)
Total L2 Elapsed Cycles [cycle]	3750744768 (+24.01%)	Total SMSP Elapsed Cycles [cycle]	27855255312 (+24.03%)

We can see that across both THREAD_TILE = 1 and 2, TILE_W = 16 is always the fastest. This matches expectation because a 16×16 tile fits cleanly into one warp's memory transaction so global memory loads become fully coalesced and shared memory banks align perfectly without modulo wrapping.

From the profiling results(baseline is TILE_W = 16 and it compared to TILE_W = 8, both are THREAD_TILE = 1), TILE_W = 8 showed noticeably lower SM and memory throughput than the baseline with TILE_W = 16. It also reports that occupancy drops because the smaller tile size increases the register pressure per block and reduces the number of blocks that can reside on the SM at the same time. This directly limits the number of active warps available to hide memory latency.





Also, Thread coarsening (THREAD_TILE = 2) didn't always improve performance. It doubles per-thread computation leading to more live variables and more registers. So, at TILE_W=16, op time 2 improved, but at TILE_W=32, the register usage exploded, cutting occupancy almost in half.

From profiling(baseline is TILE_W = 16, THREAD_TILE = 1, and it compared to TILE_W = 16, THREAD_TILE = 2), we can see THREAD_TILE = 2 pushes the SM utilization much higher (about 94% vs 69% in the baseline). This behavior is expected because each thread now computes two output rows instead of one, so the ALU and FMA pipelines stay busy longer. That means the arithmetic intensity improved. Howener, emory throughput also jumps dramatically. So memory is working harder, instead of becoming lighter.

d. Does this optimization synergize with any other optimizations? How?

Yes. This sweep-based optimization is very flexible because it only changes how we schedule work onto threads, not what we compute. So it's like an auto-tuning layer on top of other techniques

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)
   i. CUDA C++ Programming Guide, Section 5.4 "Occupancy" and Section 10.2 "Shared Memory"
   ii. The code and reference from op_6

8. Op_4: Using cuBLAS for matrix multiplication

[Google Drive link to subfolder op_4]

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

cuBLAS is a highly optimized library for matrix multiplication, and NVIDIA tunes it for the GPU architecture. The idea is to replace our handwritten matmul kernel with cuBLAS. It should gain better GEMM performance, highly optimized memory access pattern, possible use of Tensor Core–accelerated paths, and improved performance for large matrix sizes.

In theory, this method reduces the time spent on the matrix multiplication stage of convolution, but the actual benefit depends on the matrix shapes and batch size.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.
   i. Input unrolling (same as Milestone 2)

   I first unrolled the input tensor into a 2D matrix Unroll (H_unroll x W_unroll)
   ii. Doing cuBLAS GMEE

   Call the APIs from the cuBLAS library, perform the matrix multiplication.

   I used a single handle and kept it alive during the whole convolution, and used cuBLAS SGEMM to replace the shared-memory matmul kernel completely.

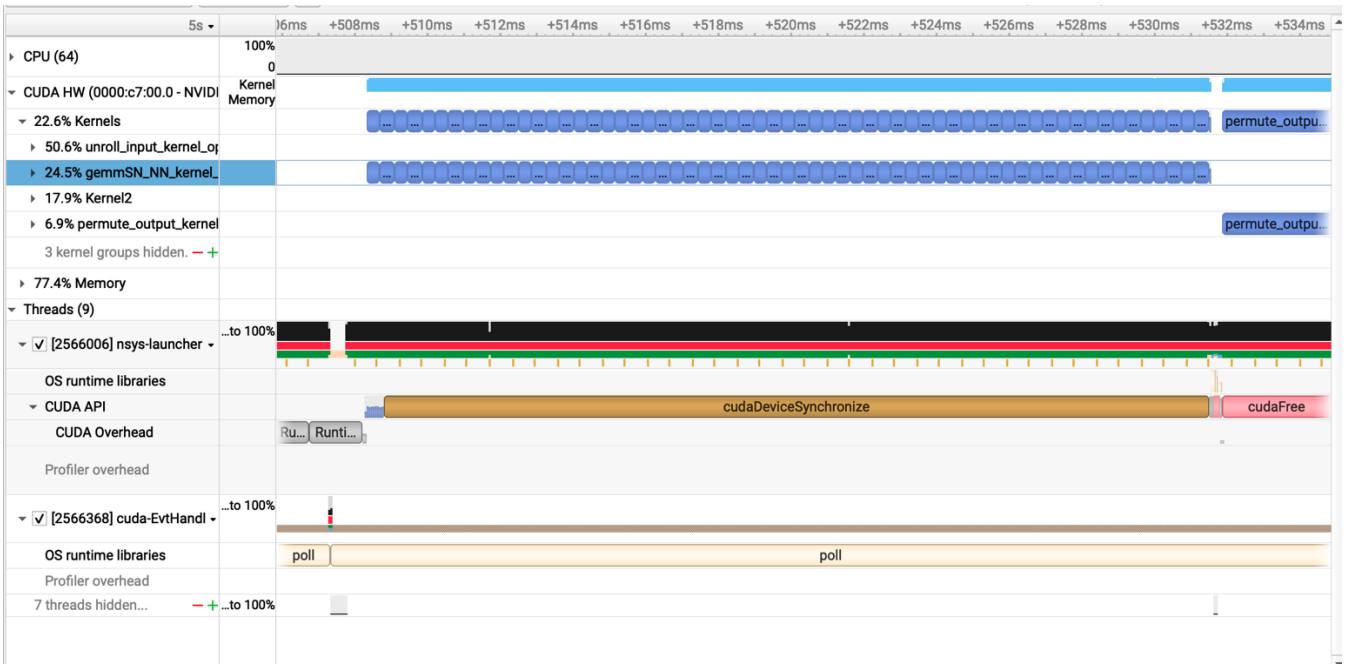
```
// cuBLAS GEMM
cublasHandle_t handle;
cublasStatus_t stat = cublasCreate(&handle);
if (stat != CUBLAS_STATUS_SUCCESS) {
    std::cerr << "cuBLAS create handle failed in " << task << std::endl;
    cudaFree(gemm_output);
    cudaFree(unrolled_matrix);
    exit(-1);
}

const float alpha = 1.0f;
const float beta  = 0.0f;

stat = cublasSgemm(handle, CUBLAS_OP_N, CUBLAS_OP_N, (int)W_unrolled, Map_out,
                   (int)H_unrolled, &alpha, unrolled_matrix, (int)W_unrolled,
                   device_mask, (int)H_unrolled, &beta, gemm_output, (int)W_unrolled
);

if (stat != CUBLAS_STATUS_SUCCESS) {
    std::cerr << "cublasSgemm failed in " << task << std::endl;
    cublasDestroy(handle);
    cudaFree(gemm_output);
    cudaFree(unrolled_matrix);
    exit(-1);
}

cublasDestroy(handle);
```

iii. Permuting the result back

After GEMM, I launched the permute kernel to reshape the output back to Batch x Map_out x Height_out x Width_out.

```
// permute
const size_t image_size = H_out * W_out;
dim3 perm_grid_dim(
    (unsigned int)((image_size + PERMUTE_BLOCK_SIZE - 1) / PERMUTE_BLOCK_SIZE), (unsigned int)Batch, 1);

permute_output_kernel_op4<<<perm_grid_dim, PERMUTE_BLOCK_SIZE>>>(
    gemm_output, device_output, Map_out, Batch, (int)image_size);

cudaFree(gemm_output);
cudaFree(unrolled_matrix);
```

ID	Estimated Speedup [%]	Function Name	Demangled Name	Duration [ms] (94.57 ms)	Runtime Improvement [ms] (26.59 ms)	Compute Throughput [%]	Memory Throughput [%]	# Registers [register/thre	Grid Size
53	36.66	gemmSN_NN_ker...	..cublasGemvTen...	0.39 (-99.25%)	0.14	22.67 (-71.87%)	93.90 (+139.66%)	96 (+159.46%)	16384, 1,
54	36.61	gemmSN_NN_ker...	..cublasGemvTen...	0.38 (-99.25%)	0.14	22.79 (-71.73%)	93.89 (+139.62%)	96 (+159.46%)	16384, 1,
55	36.70	gemmSN_NN_ker...	..cublasGemvTen...	0.39 (-99.25%)	0.14	22.71 (-71.82%)	93.92 (+139.70%)	96 (+159.46%)	16384, 1,
56	36.73	gemmSN_NN_ker...	..cublasGemvTen...	0.38 (-99.25%)	0.14	22.88 (-71.61%)	93.93 (+139.73%)	96 (+159.46%)	16384, 1,
57	36.68	gemmSN_NN_ker...	..cublasGemvTen...	0.38 (-99.25%)	0.14	22.74 (-71.79%)	94.00 (+139.89%)	96 (+159.46%)	16384, 1,
58	36.62	gemmSN_NN_ker...	..cublasGemvTen...	0.38 (-99.25%)	0.14	22.82 (-71.70%)	94.04 (+140.01%)	96 (+159.46%)	16384, 1,
59	36.67	gemmSN_NN_ker...	..cublasGemvTen...	0.38 (-99.25%)	0.14	22.79 (-71.72%)	94.11 (+140.17%)	96 (+159.46%)	16384, 1,
60	36.64	gemmSN_NN_ker...	..cublasGemvTen...	0.38 (-99.25%)	0.14	22.82 (-71.69%)	93.98 (+139.86%)	96 (+159.46%)	16384, 1,
61	36.77	gemmSN_NN_ker...	..cublasGemvTen...	0.38 (-99.25%)	0.14	22.77 (-71.75%)	94.03 (+139.99%)	96 (+159.46%)	16384, 1,
62	36.73	gemmSN_NN_ker...	..cublasGemvTen...	0.38 (-99.25%)	0.14	22.88 (-71.62%)	94.10 (+140.15%)	96 (+159.46%)	16384, 1,
63	50.00	gemmSN_NN_ker...	..cublasGemvTen...	0.02 (-99.96%)	0.01	15.81 (-80.38%)	68.35 (+74.43%)	96 (+159.46%)	607, 1,
64	7.32	permute_output_k...	permute_output_k...	3.58 (-93.01%)	0.26	12.60 (-84.37%)	92.68 (+136.54%)	40 (+8.11%)	25,10000,



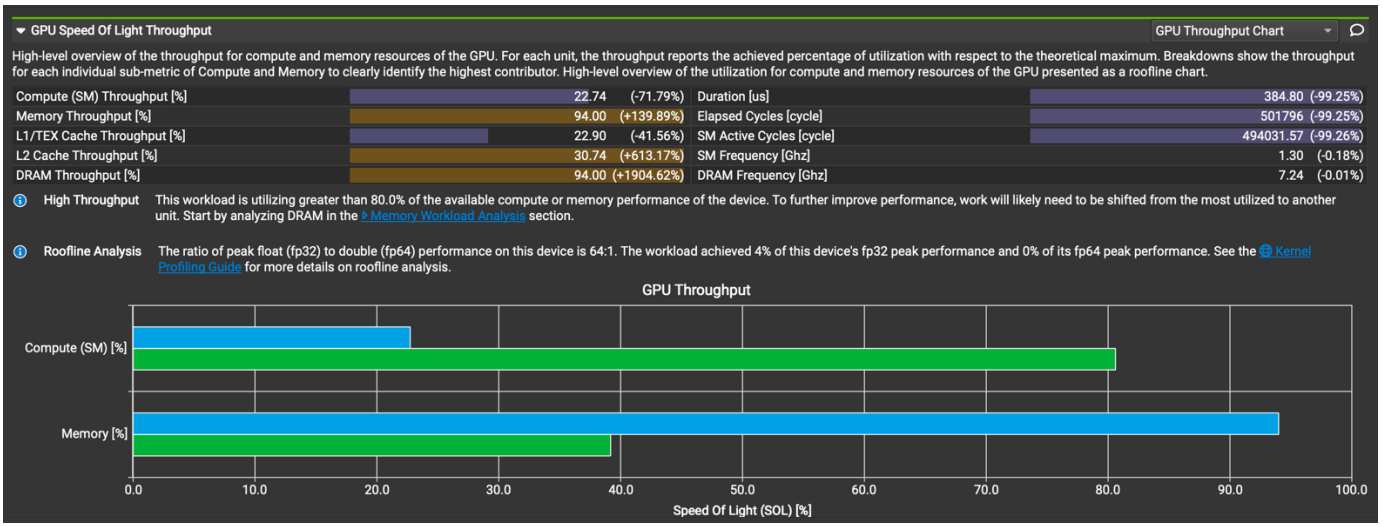
Nsight Systems and Nsight Compmute show multiple gemmSN_NN_kernel_64addr kernels, so we can confirm cuBLAS execution. Additionally, there's no invalid memory access or incorrect output pattern. Hence, it is save to say that SGEMM is correctly invoked and can replace the fused matmul.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	*49.8308 ms*	*1.28173 ms*	*2.338s*	*0.86*
1000	*54.9541 ms*	*8.91815 ms*	*9.204s*	*0.886*
10000	*94.5619 ms*	*50.6333 ms*	*1m26.682s*	*0.8714*

Not really. cuBLAS is slower than baseline for all batch sizes in OP1 (matrix multiplication).

A possible reason might be because my GEMM shape is extremely skinny. cuBLAS is optimized for large square-ish matrices, but not small–K tall-and-skinny GEMMs.



Profiling also shows memory bound. We can see memory throughput is very high and Compute throughput stays low (~22%), suggesting that cuBLAS cannot saturate compute units with this matrix shape. Also, there's too many GEMM kernel launches (number of 62) because each convolution calls cuBLAS multiple times and each SGEMM introduces fixed overhead, dominating runtime for small matrices.
But total execution time remains reasonable which is actually similar to baseline, meaning cuBLAS overhead only affects OP times but not the whole forward pass pipeline too badly.

d. Does this optimization synergize with any other optimizations? How?
cuBLAS focuses only on matrix multiplication, so it can work in synergy with other optimizations that do not target matrix multiplication, such as kernel fusion and streaming. In that case, we simply need to replace the "matrixMultiplyShared" kernel with the cuBLAS matrix multiplication APIs to integrate with this cuBLAS optimization.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)
   i. Nvidia Developer's website https://developer.nvidia.com/blog/programming-tensor-cores-cuda-9
   ii. Nvidia Official Documentation

https://docs.nvidia.com/cuda/cuda-c-programming-guide/index.html#warp-matrix-functions

9. Op_5: Fixed point (FP16) arithmetic implementation

[Google Drive link to subfolder op_5]

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

Using FP16 should reduce memory traffic and improve overall speed, because FP16 values are half the size of FP32. When using __half2, two FP16 values fit inside one 32-bit register, which can improve memory efficiency and allow the GPU to do two FP16 multiplications at once.

In practice, the main expected benefits are faster arithmetic using FP16 hardware paths, lower shared-memory footprint, better ALU utilization, and slightly better memory efficiency inside the kernel.

Since the convolution is a large matrix multiplication, using FP16 usually helps reduce the total time for accumulation inside each tile. Accuracy is expected to remain the same because the original convolution for Fashion-MNIST is not very sensitive to reduced precision.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

i. Store tiles in shared memory as __half2

```
__shared__ __half2 shMask[TILE_WIDTH][TILE_WIDTH];
__shared__ __half2 shInput[TILE_WIDTH][TILE_WIDTH];
```

ii. Load FP32 data from global memory, convert to FP16, and pack into __half2

I read the input and mask as FP32 as usual to keep the accuracy (same as baseline).

Since the second lane is zero, this keeps the math simple but still runs FP16 arithmetic.

```
// load mask tile
float m_val = 0.0f;
if (row < Map_out && k_index_for_mask < H_unroll) {
    int c_idx  = k_index_for_mask / (K * K);
    int rem    = k_index_for_mask % (K * K);
    int p_idx  = rem / K;
    int q_idx  = rem % K;
    m_val = MASK_4D(row, c_idx, p_idx, q_idx);
}
shMask[ty][tx] = __halves2half2(__float2half(m_val), __float2half(0.0f));

// load input tile
float x_val = 0.0f;
if (col < W_unroll && k_index_for_input < H_unroll) {
    int c_idx  = k_index_for_input / (K * K);
    int rem    = k_index_for_input % (K * K);
    int p_idx  = rem / K;
    int q_idx  = rem % K;

    int h_in = h_out + p_idx;
    int w_in = w_out + q_idx;

    x_val = IN_4D(b, c_idx, h_in, w_in);
}
shInput[ty][tx] = __halves2half2(__float2half(x_val), __float2half(0.0f));
```

iii. Perform FP16 mul + FP32 accumulate

This matches the original FP32 output mathematically, and also ensures no accuracy loss because accumulation is still performed in FP32.

```
// compute partial dot product for this tile
if (row < Map_out && col < W_unroll) {
    #pragma unroll
    for (int k = 0; k < TILE_WIDTH; ++k) {
        __half2 prod = __hmul2(shMask[ty][k], shInput[k][tx]);
        float2 prod_f = __half22float2(prod);
        acc += prod_f.x + prod_f.y;
    }
}
```

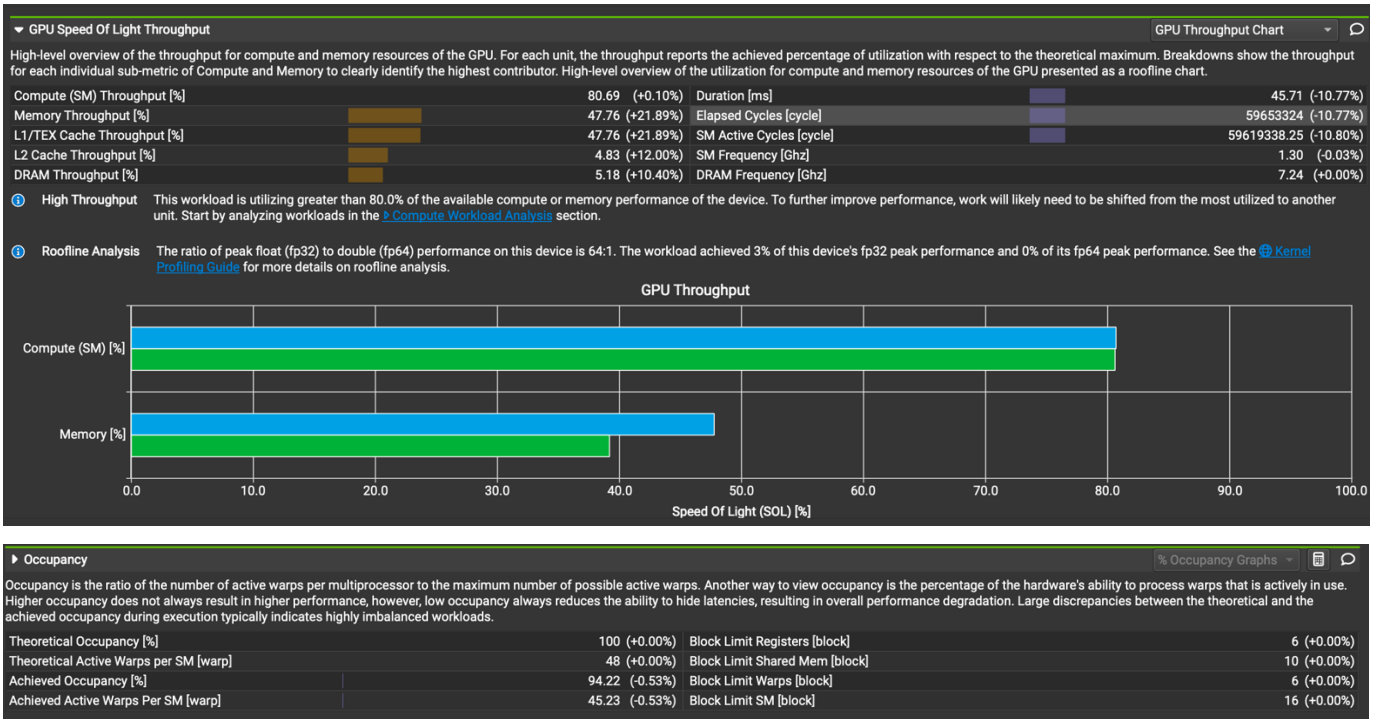
iv. Return result as FP32

```
if (row < Map_out && col < W_unroll) {
    int hw = col % (H_out * W_out);
    int h  = hw / W_out;
    int w  = hw % W_out;
    output[b * Map_out * H_out * W_out + row * H_out * W_out + h * W_out + w] = acc;
}
```

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.465763 ms	0.32034 ms	1.427s	0.86
1000	3.53522 ms	3.04506 ms	9.745s	0.886
10000	34.7259 ms	30.1922 ms	1m26.192s	0.8714

Yes, the performance matched the expectation for a partially FP16 implementation. Compared to the kernel fusion baseline, we can see op times are consistently lower and accuracy remains the same. Also, total execution time is slightly faster, which matches the expected behavior of reduced memory pressure.



Even though FP16 uses smaller data, my implementation still loads global memory as FP32, then converts to FP16 only when placing data into shared memory.

So bytes loaded from global memory didn't shrink, but compute became faster (FP16 math), leading to a decrease in runtime.

Also, we can see from the profiling that compute throughput is almost the same, DRAM bandwidth remained normal, and occupancy remained unchanged. This

matches the idea that the compute loop became faster, creating higher pressure (but still valid pressure) on memory pipelines.

d. Does this optimization synergize with any other optimizations? How?
   Yes. FP16 mainly reduces memory bottlenecks, so it works well when paired with any optimization that pushes compute efficiency. By using FP16 to reduce memory throughput, it complements tensor cores, enhancing compute throughput by efficiently handling mixed-precision operations. They address memory bottlenecks while improving computational efficiency. Optimizations like loop unrolling, constant memory, the restrict keyword, and CUDA streams can also further enhance performance.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)
   i. Mixed-Precision Programming with CUDA 8
      https://developer.nvidia.com/blog/mixed-precision-programming-cuda-8/
   ii. Type __half
      https://docs.nvidia.com/cuda/cuda-math-api/cuda_math_api/struct____half.html#_CPPv46__half
   iii. Half Arithmetic Functions
      https://docs.nvidia.com/cuda/cuda-math-api/cuda_math_api/group__CUDA__MATH____HALF__ARITHMETIC.html
   iv. Type __half2
      https://docs.nvidia.com/cuda/cuda-math-api/cuda_math_api/struct____half2.html
   v. Half2 Arithmetic Functions
      https://docs.nvidia.com/cuda/cuda-math-api/cuda_math_api/group__CUDA__MATH____HALF2__ARITHMETIC.html
   vi. Half Precision Conversion and Data Movement
      https://docs.nvidia.com/cuda/cuda-math-api/cuda_math_api/group__CUDA__MATH____HALF__MISC.html

10. Op_6: Using Joint Register and Shared Memory Tiling to speed up matrix multiplication

[Google Drive link to subfolder op_6]

a. How does this optimization theoretically optimize your convolution kernel? Expected behavior?

This optimization lets each thread compute two output rows instead of one by using row tiling. The goal is to increase arithmetic intensity: once a thread loads data from shared memory, using the same loaded tile to compute two output rows improves reuse without increasing global memory traffic.

In theory, each thread should perform more useful work, which can improve ALU/FMA utilization. Shared memory tiles get reused twice, which reduces effective shared-memory pressure per output. The kernel launches fewer total threads, lowering scheduling overhead. Global memory traffic stays the same, so the kernel becomes more compute-bound instead of memory-bound.

We expect slightly better runtime than the baseline because more work is done per thread while memory cost remains unchanged.

b. How did you implement your code? Explain thoroughly and show code snippets. Justify the correctness of your implementation with proper profiling results.

i. Load two mask and input rows per thread into shared memory

```
// load TILE of mask: ROW_TILE rows per thread
#pragma unroll
for (int r = 0; r < ROW_TILE; ++r) {
    int gRow = baseRow + r;
    int shRow = ty * ROW_TILE + r;

    if (gRow < Map_out && k_col < H_unroll) {
        int c  = k_col / (K * K);
        int kk = k_col % (K * K);
        int p  = kk / K;
        int q  = kk % K;
        shMask[shRow][tx] = MASK_4D(gRow, c, p, q);
    } else {
        shMask[shRow][tx] = 0.0f;
    }
}
```

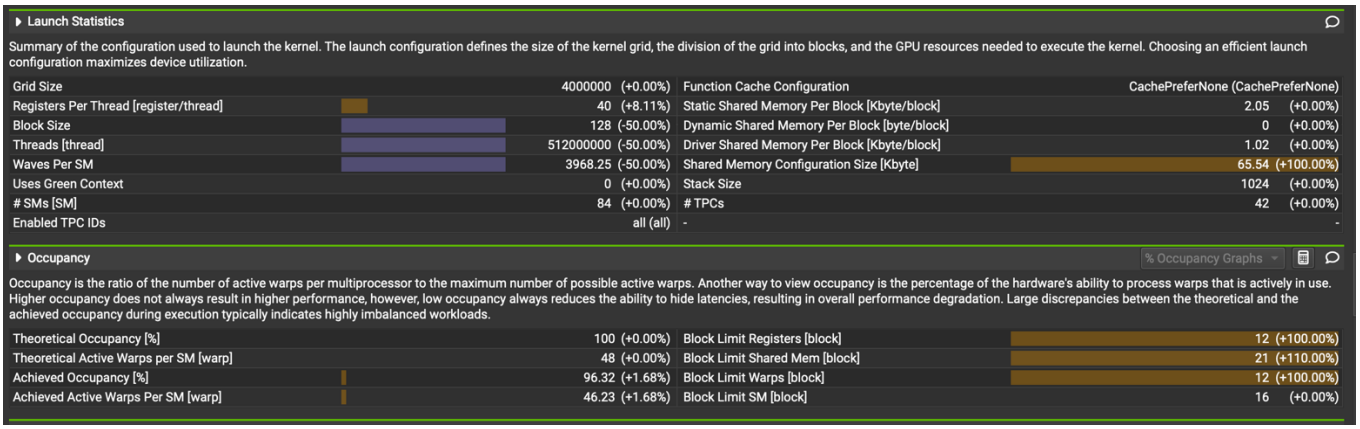
```
// load TILE of input: also ROW_TILE rows per thread
#pragma unroll
for (int r = 0; r < ROW_TILE; ++r) {
    int k_row = k_row_base + r;
    int shRow = ty * ROW_TILE + r;

    if (col < W_unroll && k_row < H_unroll) {
        int b    = col / (H_out * W_out);
        int hw   = col % (H_out * W_out);
        int h_o  = hw / W_out;
        int w_o  = hw % W_out;

        int c  = k_row / (K * K);
        int kk = k_row % (K * K);
        int p  = kk / K;
        int q  = kk % K;

        int h = h_o + p;
        int w = w_o + q;

        shInput[shRow][tx] = IN_4D(b, c, h, w);
    } else {
        shInput[shRow][tx] = 0.0f;
    }
}
```

ii. Compute both rows inside the same loop

```
// compute partial sums from this tile
for (int k = 0; k < TILE_WIDTH; ++k) {
    float in_val = shInput[k][tx];
    #pragma unroll
    for (int r = 0; r < ROW_TILE; ++r) {
        int shRow = ty * ROW_TILE + r;
        acc[r] += shMask[shRow][k] * in_val;
    }
}
```

iii. Store both results

```
// write back both rows this thread is responsible for
#pragma unroll
for (int r = 0; r < ROW_TILE; ++r) {
    int gRow = baseRow + r;
    if (gRow < Map_out && col < W_unroll) {
        int b  = col / (H_out * W_out);
        int hw = col % (H_out * W_out);
        int h  = hw / W_out;
        int w  = hw % W_out;

        output[b * Map_out * H_out * W_out + gRow * H_out * W_out + h * W_out + w] = acc[r];
    }
}
```



Launch Statistics

Summary of the configuration used to launch the kernel. The launch configuration defines the size of the kernel grid, the division of the grid into blocks, and the GPU resources needed to execute the kernel. Choosing an efficient launch configuration maximizes device utilization.

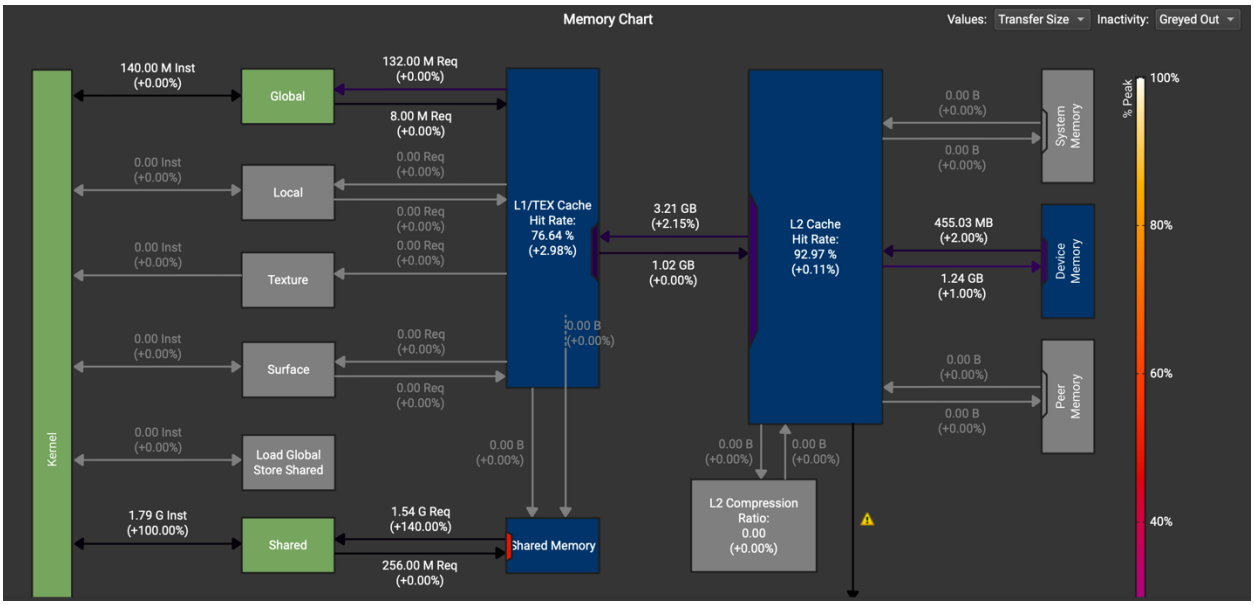
Grid Size	4000000 (+0.00%)	Function Cache Configuration	CachePreferNone (CachePreferNone)
Registers Per Thread [register/thread]	40 (+8.11%)	Static Shared Memory Per Block [Kbyte/block]	2.05 (+0.00%)
Block Size	128 (-50.00%)	Dynamic Shared Memory Per Block [byte/block]	0 (+0.00%)
Threads [thread]	512000000 (-50.00%)	Driver Shared Memory Per Block [Kbyte/block]	1.02 (+0.00%)
Waves Per SM	3968.25 (-50.00%)	Shared Memory Configuration Size [Kbyte]	65.54 (+100.00%)
Uses Green Context	0 (+0.00%)	Stack Size	1024 (+0.00%)
# SMs [SM]	84 (+0.00%)	# TPCs	42 (+0.00%)
Enabled TPC IDs	all (all)	-	-

Occupancy — % Occupancy Graphs

Occupancy is the ratio of the number of active warps per multiprocessor to the maximum number of possible active warps. Another way to view occupancy is the percentage of the hardware's ability to process warps that is actively in use. Higher occupancy does not always result in higher performance, however, low occupancy always reduces the ability to hide latencies, resulting in overall performance degradation. Large discrepancies between the theoretical and the achieved occupancy during execution typically indicates highly imbalanced workloads.

Theoretical Occupancy [%]	100 (+0.00%)	Block Limit Registers [block]	12 (+100.00%)
Theoretical Active Warps per SM [warp]	48 (+0.00%)	Block Limit Shared Mem [block]	21 (+110.00%)
Achieved Occupancy [%]	96.32 (+1.68%)	Block Limit Warps [block]	12 (+100.00%)
Achieved Active Warps Per SM [warp]	46.23 (+1.68%)	Block Limit SM [block]	16 (+0.00%)

From Nsight Compute, we can see that registers per thread increased from 36 → 40 (+8.11%). This is expected because each thread now maintains two accumulators (acc[2]) and additional indexing logic. The increase is small, and the kernel still reaches achieved occupancy 96.32% and achieved active warps per SM: 46.23 / 48. This proves the register pressure is healthy and the kernel is not constrained by occupancy.

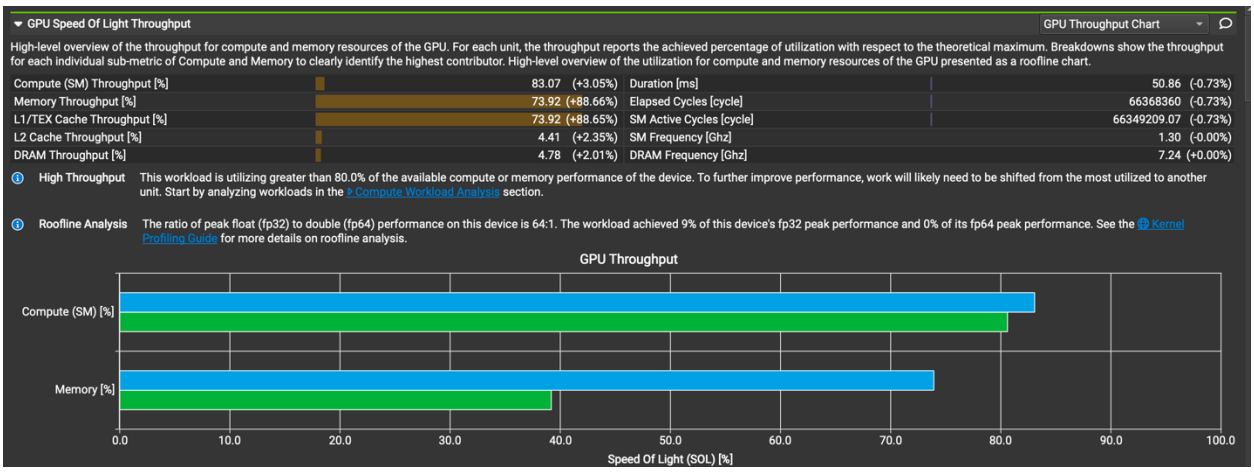


We can also see that Shared memory activity increased (+140%). This matches the design where each thread loads two rows instead of one. However, shared memory is still faster than global memory, and profiler shows no shared-memory replay events and 256 MB shared mem requests. This confirms the shared memory tiling is functioning correctly.

c. Did the performance match your expectation? Explain why or why not, by analyzing profiling results.

Batch Size	Op Time 1	Op Time 2	Total Execution Time	Accuracy
100	0.481972 ms	0.270316 ms	2.167s	0.86
1000	3.91429 ms	2.53156 ms	9.435s	0.886
10000	38.6653 ms	25.0375 ms	1m31.062s	0.8714

We can see that op time 1 is slightly faster than baseline and op time 2 is much faster (≈ 30% improvement) while accuracy is unchanged, though total execution time is similar or slightly higher (likely due to host scheduling or memcpy variations).



According to the Nsight Compute, it is shown that L1/TEX cache hit rate increased. This is because each shared memory tile is reused twice, leading to global memory requests becoming more coherent. The higher hit rate confirms improved spatial locality and correct reuse of loaded tiles.

d. Does this optimization synergize with any other optimizations? How?

   Yes. It works well with several previous techniques. For instance, using FP16 together with ROW_TILE would multiply the benefit because shared-memory traffic would shrink even further.

   This optimization is a good option that improves compute efficiency and pairs well with techniques that reduce memory traffic.

e. List your references used while implementing this technique. (you must mention textbook pages at the minimum)

   i. ECE 508 Lecture Slides on Joint Register and Shared Memory Tiling
      https://lumetta.web.engr.illinois.edu/508/slides/lecture4.pdf

   ii. The Guest Profiling Lecture
      https://carlpearson.net/pdf/20200416_nsight.pdf